
Linear Regression

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Slides that adapt figures or content from other sources have a link to the source.

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Outline

- Linear Regression Formalization
- Closed form solution
- Gradient descent solution
- Mini-batch gradient descent
- Stability in gradient descent and feature scaling
- Polynomial (Linear) Regression
- Regularization
- Summary

Formalization

Recall Helpful Matrix Notation

- We can represent a dataset $\{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, y_m)\}$ with m examples and n features as a feature matrix $X \in \mathbb{R}^{m \times n}$ and a label vector $Y \in \mathbb{R}^m$

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_m^T \end{bmatrix} \quad \bigg| \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

- X is referred to as the feature or data matrix. Y is the label or output vector.

Formalism and Notation [1]

- We have a regression task with instances (x_i, y_i) where $x_i \in \mathbb{R}^n$ is a vector with n features and $y_i \in \mathbb{R}$ is a continuous output label.
- Note that we will denote x_{ij} as feature j of instance i . So, $x_{3,9}$ refers to 9th feature value of 3rd instance in our dataset.
- In linear regression, we **assume** that the y_i label can be approximated by a *linearly weighted* combination of the input features. That is:

$$\begin{aligned}\hat{y}_i &= h(x_i) \\ &= w_0 1 + w_1 x_{i1} + w_2 x_{i2} + \dots + w_n x_{in}\end{aligned}$$

- w_0 is the intercept (also called the bias), and w_j for $j = 1, 2, \dots, n$ is the weight given to each input feature j .
- This is called a linear or linearly weighted combination because it is linear in the weight parameters.

Formalism and Notation [2]

- To make notation simpler, we add a 1 to the beginning of our instance vector x_i . This is to account for the intercept weight w_0

$$x_i = \begin{bmatrix} 1 \\ x_{i1} \\ x_{i2} \\ \vdots \\ x_{in} \end{bmatrix}$$

A vector is by default a column vector and is written in a vertical manner.

- (From linear algebra) The transpose of a column vector v is denoted as v^T . So, the transpose of an instance vector x_i is:

$$x_i^T = [1 \quad x_{i1} \quad x_{i2} \quad \dots \quad x_{in}]$$

The transpose a vector is written in a horizontal manner

Formalism and Notation [3]

- So now we can write our hypothesis function as a **dot product** of two vectors, namely the instance vector x_i and the weights vector w :

$$\begin{aligned}\hat{y}_i &= h(x_i) \\ &= w_0 1 + w_1 x_{i1} + w_2 x_{i2} + \dots + w_n x_{in} \\ &= x_i^T w\end{aligned}$$

$$= \begin{bmatrix} 1 & x_{i1} & x_{i2} & \dots & x_{in} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

Be sure to review:

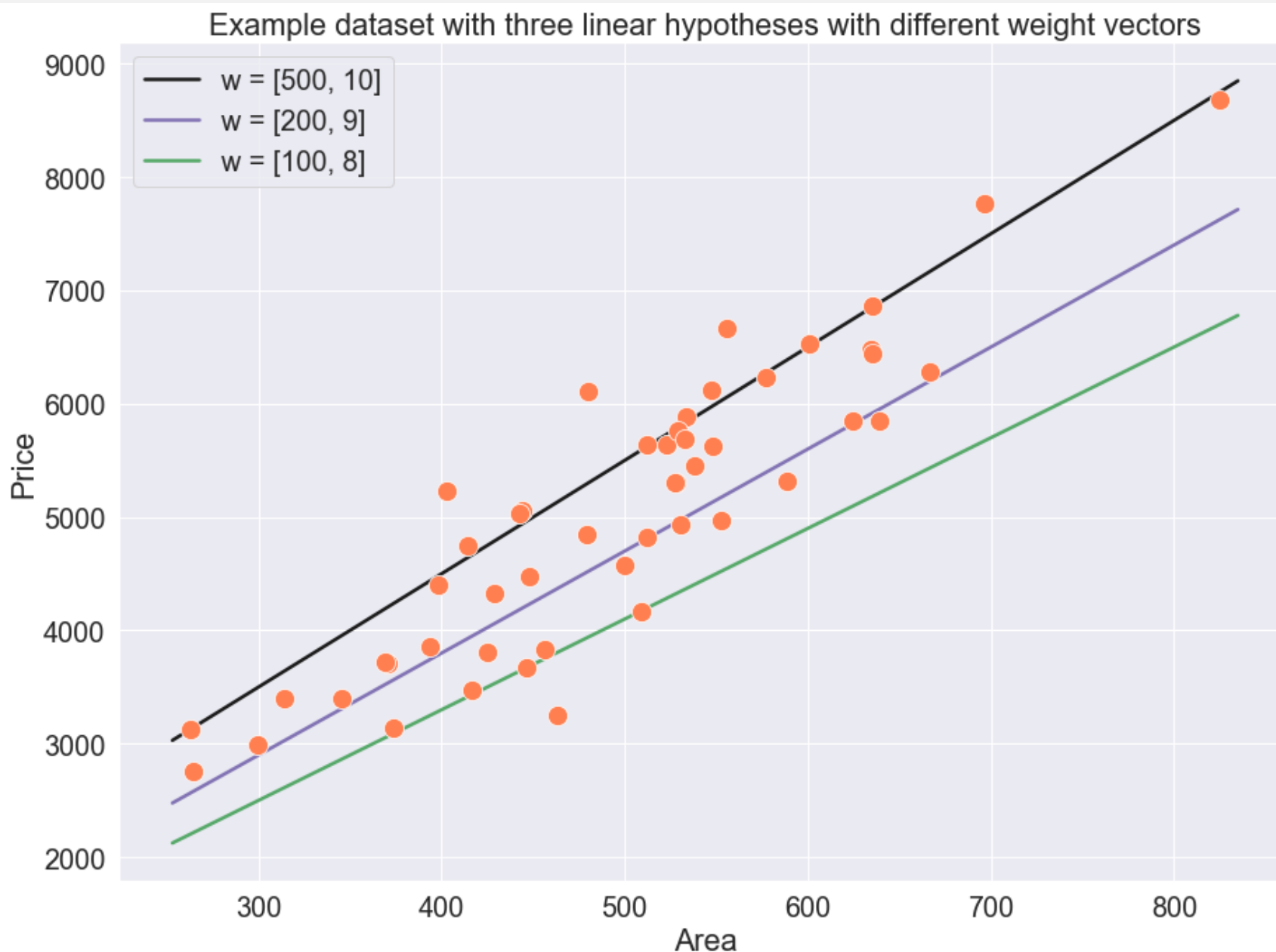
- Vector-vector dot product
- Vector-vector multiplication
- Matrix-vector multiplication
- Matrix-matrix multiplication

Formalism and Notation [4]

- What is the hypothesis space/set of linear regression? It is the set of all linear weighted hypotheses:

$$\mathcal{H} = \{h(x_i; w) = x_i^T w \quad | \quad w \in \mathbb{R}^{n+1}\}$$

- For a hypothesis written in this notation $h(x; w)$ with semicolon (;), we say that h takes input x and is *parameterized* by w .
- Also note that the above hypothesis space is considering linear regression with intercept. We can also do linear regression without intercept which means that the hypothesis space considers only n weight parameters.

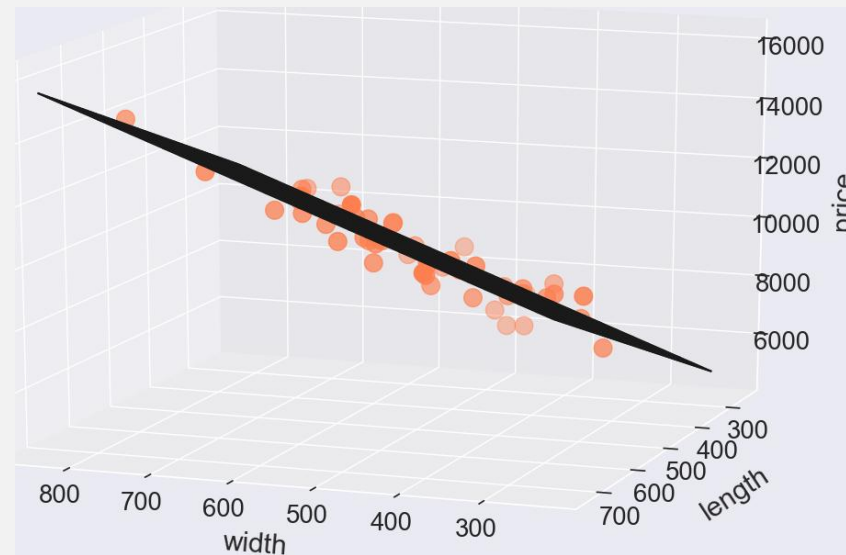
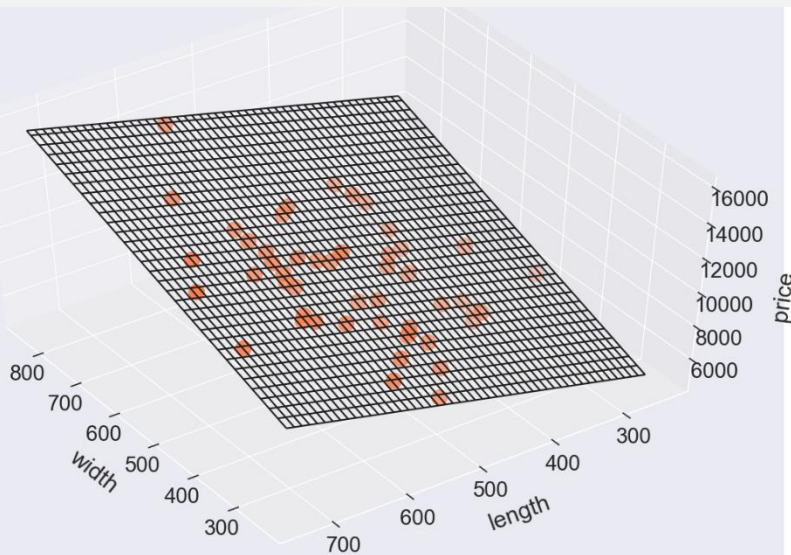


- Example of a dataset with one feature (Area) and output (price).
- Three linear hypotheses are shown with different weight vectors.

$$\hat{y} = w_0 + w_1 Area$$

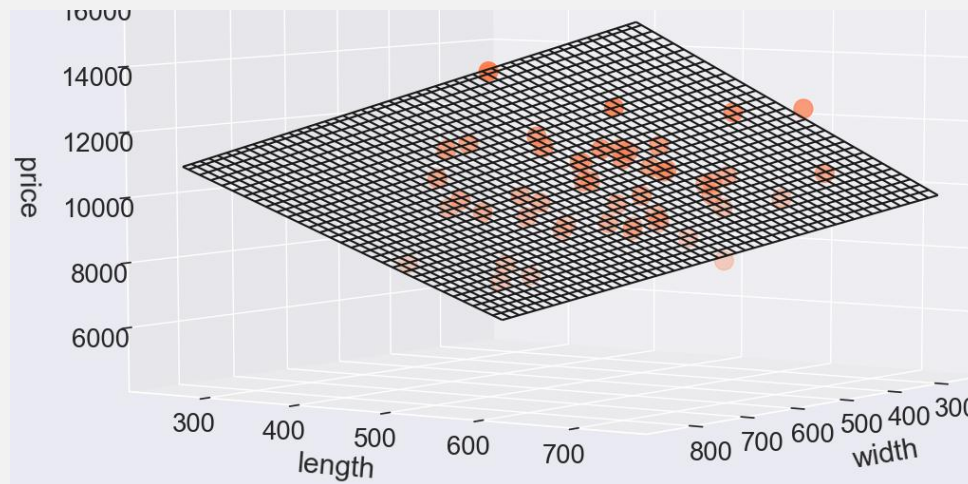
- There are infinitely many linear hypotheses since there are infinitely many values for the weight vector.

Linear Regression – Example Two-feature Problem



- Example of a dataset with two features (length, width) and output (price).
- The same 2D plane hypothesis is shown from different perspectives.

$$\hat{y} = w_0 + w_1 \text{width} + w_2 \text{length}$$



- For a 1-feature problem, the hypothesis is a 1D line. For a 2-feature problem, the hypothesis is a 2D plane.
- For an n -feature problem, the hypothesis is an n D surface (also called [hyperplane](#)).

Helpful Matrix Notation

- For ML algorithms that use an intercept weight (linear regression, logistic regression, neural networks). We can add a column of 1s to account for the intercept weight.

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & \dots & x_{2n} \\ & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_m^T \end{bmatrix} \quad \bigg| \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

Helpful Matrix Notation

- Suppose that we have a weight vector $w \in \mathbb{R}^{n+1}$, then we can compute the predictions on $x_1, x_2, \dots, x_m$ as follows:

$$\hat{Y} = Xw = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & \dots & x_{2n} \\ & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} = \begin{bmatrix} w_0 1 + w_1 x_{11} + w_2 x_{12} + \dots + w_n x_{1n} \\ w_0 1 + w_1 x_{21} + w_2 x_{22} + \dots + w_n x_{2n} \\ \vdots \\ w_0 1 + w_1 x_{m1} + w_2 x_{m2} + \dots + w_n x_{mn} \end{bmatrix}$$

Helpful Matrix Notation

- Suppose that we have a weight vector $w \in \mathbb{R}^{n+1}$, then we can compute the predictions on $x_1, x_2, \dots, x_m$ as follows:

$$\hat{Y} = Xw = \begin{bmatrix} w_0 1 + w_1 x_{11} + w_2 x_{12} + \cdots + w_n x_{1n} \\ w_0 1 + w_1 x_{21} + w_2 x_{22} + \cdots + w_n x_{2n} \\ \vdots \\ w_0 1 + w_1 x_{m1} + w_2 x_{m2} + \cdots + w_n x_{mn} \end{bmatrix} = \begin{bmatrix} \hat{y}_1 = x_1^T w \\ \hat{y}_2 = x_2^T w \\ \vdots \\ \hat{y}_m = x_m^T w \end{bmatrix}$$

Helpful Notation

- (From linear algebra) the euclidean norm (or 2-norm) of a vector a is denoted as $\|a\|_2$. It is the square root of the sum of squares of the elements in a :

$$\|a\|_2 = \sqrt{\sum_i a_i^2} \implies \|a\|_2^2 = \sum_i a_i^2 = a^T a$$

- Thus, given two vectors a and b , then:

$$\|a - b\|_2^2 = \sum_i (a_i - b_i)^2 = (a - b)^T (a - b)$$

Finding best hypothesis in Linear Regression

- Given a training set $\{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ with m examples and n features.
- In linear regression, we consider the hypothesis space of linear weighted features:

$$\mathcal{H} = \{h(x_i; w) = x_i^T w \mid w \in \mathbb{R}^{n+1}\}$$

- How do we find the best w that fits the training data?
- There are two popular approaches for linear regression:
 - Least-squares minimization. This is a straightforward and intuitive optimization formulation.
 - Statistical formulation. We assume distributions and estimate parameters of those distributions. This is a flexible formulation, but requires probability background (which we have covered).
 - Both approaches reach the same optimization formula, and so, yield the same solution.

Least-squares minimization [1]

- Given a training set $\{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ with m examples and n features.
- In least-squares minimization, we use the squared loss. For a given dataset and $w \in \mathbb{R}^{n+1}$, the squared loss is:

$$\begin{aligned}\ell(w) &= \sum_{i=1}^m (y_i - \hat{y}_i)^2 = \sum_{i=1}^m (y_i - x_i^T w)^2 \\ &= \|Y - \hat{Y}\|_2^2 = \|Y - Xw\|_2^2 = (Y - Xw)^T (Y - Xw)\end{aligned}$$

Least-squares minimization [2]

- The goal is to find $w \in \mathbb{R}^{n+1}$ that minimizes the squared loss between the true labels y_i and the predicted labels $\hat{y}_i$ for the given dataset $\{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$.

$$\begin{aligned} w^* &= \arg \min_{w \in \mathbb{R}^{n+1}} \ell(w) \\ &= \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - \hat{y}_i)^2 \\ &= \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - x_i^T w)^2 \\ &= \arg \min_{w \in \mathbb{R}^{n+1}} \|Y - Xw\|_2^2 \\ &= \arg \min_{w \in \mathbb{R}^{n+1}} (Y - Xw)^T (Y - Xw) \end{aligned}$$

Statistical Formulation [1]

- In the statistical approach, we formulate the linear regression problem as follows:
 - The features x_i are generated by some process, and we do not care about modelling this process.
 - The labels y_i are constructed as $y_i = x_i^T w + \epsilon$
 - ϵ is called the error, residual, or noise. It is assumed to follow the distribution $\epsilon \sim \text{Normal}(0, \sigma^2)$
- So, what does this modelling mean? We have $y_i = x_i^T w + \epsilon$ where $\epsilon \sim \text{Normal}(0, \sigma^2)$
 - This tells us that the label is set by a linear combination of the features *plus* some noise quantity that makes it deviate from the linear plane.
 - Since $\epsilon \sim \text{Normal}(0, \sigma^2)$, then $(x_i^T w + \epsilon) \sim \text{Normal}(x_i^T w, \sigma^2)$. Thus, $y_i | x_i \sim \text{Normal}(x_i^T w, \sigma^2)$
- The illustration shows this for a 1-feature problem.
 - The line determines the starting location of $y = x^T w$
 - Then noise ϵ is added to randomly push y up or down. This explains the deviation from the linear plane.
- Linear regression models $y_i | x_i$ only. Thus, it is a **discriminative** model.

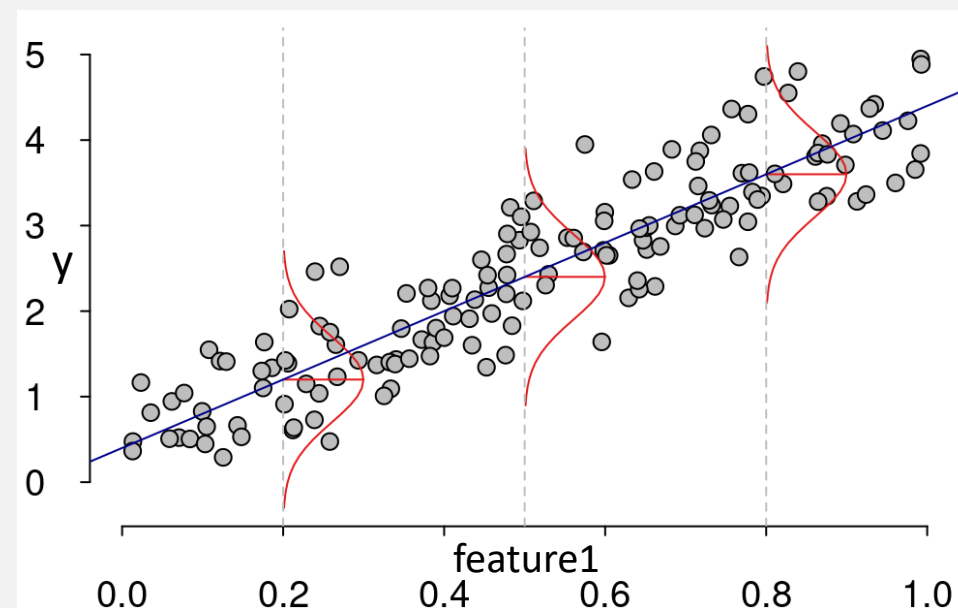


Image from [link](#)

Statistical Formulation [2]

- With statistical formulation we have $y_i = x_i^T w + \epsilon$ where $\epsilon \sim \text{Normal}(0, \sigma^2)$ and $y_i \mid x_i \sim \text{Normal}(x_i^T w, \sigma^2)$
- How to find best w with this formulation?
- We can use MLE to estimate the parameters w of this model. In MLE, the goal is to find the parameter setting for w that maximizes this probability of the training data.

$$w^* = \arg \max_{w \in \mathbb{R}^{n+1}} \text{Prob}(\text{Train})$$

$$= \arg \max_{w \in \mathbb{R}^{n+1}} \text{Prob}[(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)]$$

$$= \arg \max_{w \in \mathbb{R}^{n+1}} \prod_{i=1}^m \text{Prob}(x_i, y_i)$$

Using independent and identically distributed (i.i.d) assumption.

$$= \arg \max_{w \in \mathbb{R}^{n+1}} \prod_{i=1}^m \text{Prob}(y_i \mid x_i) \text{Prob}(x_i) = \arg \max_{w \in \mathbb{R}^{n+1}} \prod_{i=1}^m \text{Prob}(y_i \mid x_i)$$

We can drop $\text{Prob}(x_i)$ because it doesn't depend on w .

Statistical Formulation [3]

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

- How to find best w with this formulation?

$$w^* = \arg \max_{w \in \mathbb{R}^{n+1}} \text{Prob}(\text{Train}) = \arg \max_{w \in \mathbb{R}^{n+1}} \prod_{i=1}^m \text{Prob}(y_i | x_i)$$

- With statistical formulation we have $y_i = x_i^T w + \epsilon$ where $\epsilon \sim \text{Normal}(0, \sigma^2)$ and $y_i | x_i \sim \text{Normal}(x_i^T w, \sigma^2)$
- So, to continue plug in the normal distribution PDF with $\mu = x_i^T w$ and σ .

$$w^* = \arg \max_{w \in \mathbb{R}^{n+1}} \prod_{i=1}^m \text{Prob}(y_i | x_i)$$

$$= \arg \max_{w \in \mathbb{R}^{n+1}} \prod_{i=1}^m \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2} \left(\frac{y_i - x_i^T w}{\sigma} \right)^2}$$

We can drop $\frac{1}{\sigma\sqrt{2\pi}}$ because it doesn't depend on w .

$$= \arg \max_{w \in \mathbb{R}^{n+1}} \prod_{i=1}^m e^{-\frac{1}{2} \left(\frac{y_i - x_i^T w}{\sigma} \right)^2}$$

Statistical Formulation [4]

$$w^* = \arg \max_{w \in \mathbb{R}^{n+1}} \text{Prob}(\text{Train}) = \arg \max_{w \in \mathbb{R}^{n+1}} \prod_{i=1}^m \text{Prob}(y_i | x_i)$$

$$= \arg \max_{w \in \mathbb{R}^{n+1}} \prod_{i=1}^m e^{-\frac{1}{2} \left(\frac{y_i - x_i^T w}{\sigma} \right)^2}$$

Apply log

$$= \arg \max_{w \in \mathbb{R}^{n+1}} -\frac{1}{2\sigma^2} \sum_{i=1}^m (y_i - x_i^T w)^2$$

Drop $\frac{1}{2\sigma^2}$

$$= \arg \max_{w \in \mathbb{R}^{n+1}} -\sum_{i=1}^m (y_i - x_i^T w)^2$$

Multiply by -1 to change maximization into minimization

$$= \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - x_i^T w)^2 = \arg \min_{w \in \mathbb{R}^{n+1}} \|Y - Xw\|_2^2 = \arg \min_{w \in \mathbb{R}^{n+1}} (Y - Xw)^T (Y - Xw)$$

- This is the same as least-squares minimization! So, both approaches lead to the same minimization formula.

Statistical Formulation [5]

- Suppose that we trained linear regression and found w^* .
- With statistical formulation we have $y_i = x_i^T w^* + \epsilon$ where $\epsilon \sim \text{Normal}(0, \sigma^2)$ and $y_i \mid x_i \sim \text{Normal}(x_i^T w^*, \sigma^2)$

$$\begin{aligned} w^* &= \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - x_i^T w)^2 \\ &= \arg \min_{w \in \mathbb{R}^{n+1}} \|Y - Xw\|_2^2 \\ &= \arg \min_{w \in \mathbb{R}^{n+1}} (Y - Xw)^T (Y - Xw) \end{aligned}$$

- Given x_{new} , what should be the prediction label $\hat{y}_{new}$ given x_{new} ?
- Since $y \mid x \sim \text{Normal}(x^T w^*, \sigma^2)$, the **most probable** value for y would be the mean (expectation).

$$\hat{y} = \mathbb{E}[y] = x^T w^*$$

- This is the same prediction we used for the least-squared minimization formulation.

Statistical Formulation [5]

- Suppose that we trained linear regression and found w^* .
- With statistical formulation we have $y_i = x_i^T w^* + \epsilon$ where $\epsilon \sim \text{Normal}(0, \sigma^2)$ and $y_i \mid x_i \sim \text{Normal}(x_i^T w^*, \sigma^2)$

$$\begin{aligned} w^* &= \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - x_i^T w)^2 \\ &= \arg \min_{w \in \mathbb{R}^{n+1}} \|Y - Xw\|_2^2 \\ &= \arg \min_{w \in \mathbb{R}^{n+1}} (Y - Xw)^T (Y - Xw) \end{aligned}$$

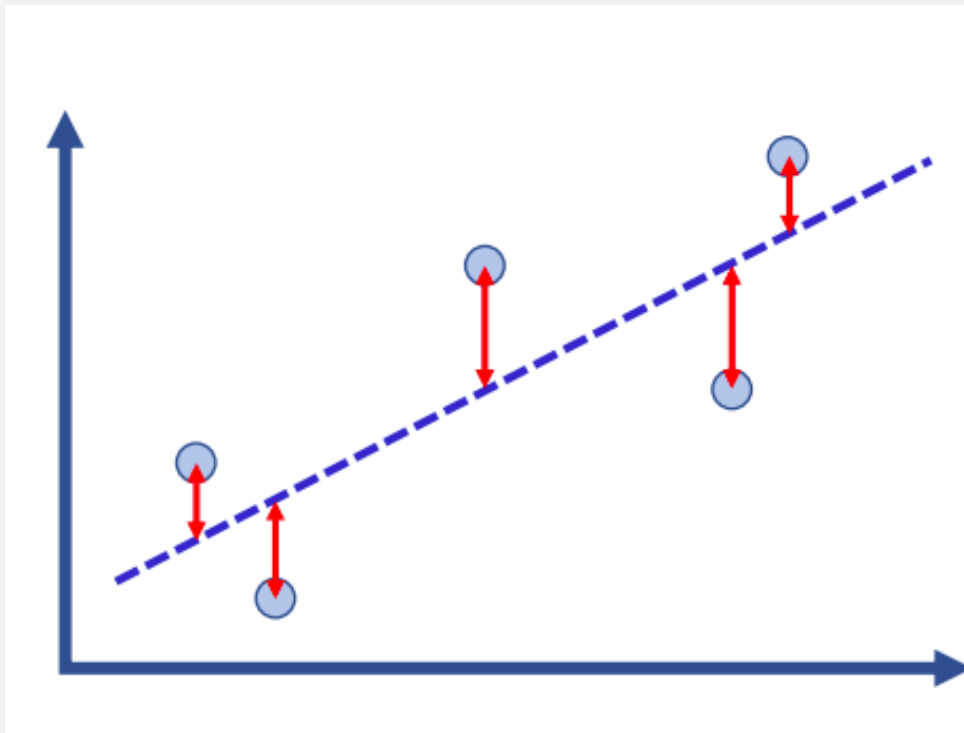
- Given x_{new} , what should be the prediction label $\hat{y}_{new}$ given x_{new} ?
- Since $y \mid x \sim \text{Normal}(x^T w^*, \sigma^2)$, the **most probable** value for y would be the mean (expectation).

$$\hat{y} = \mathbb{E}[y] = x^T w^*$$

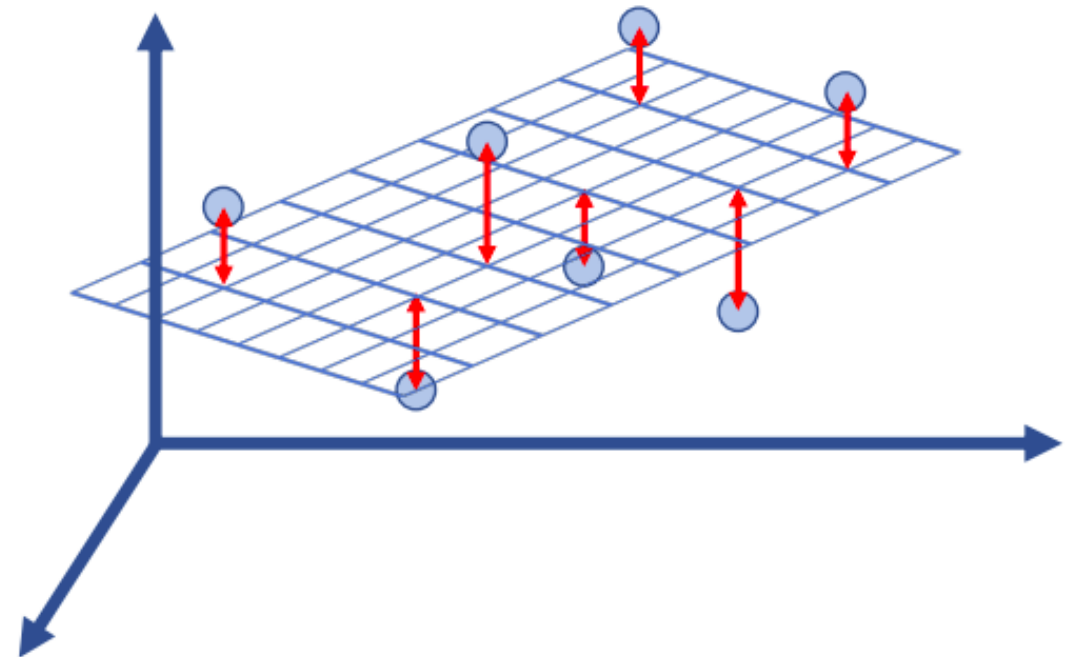
- This is the same prediction we used for the least-squared minimization formulation.

Linear Regression Fitting Visualized

- Visually, minimizing the squared loss is finding the linear surface that minimizes the *vertical* distances between the datapoints and their prediction on the linear surface.



One-feature problem fitted by line



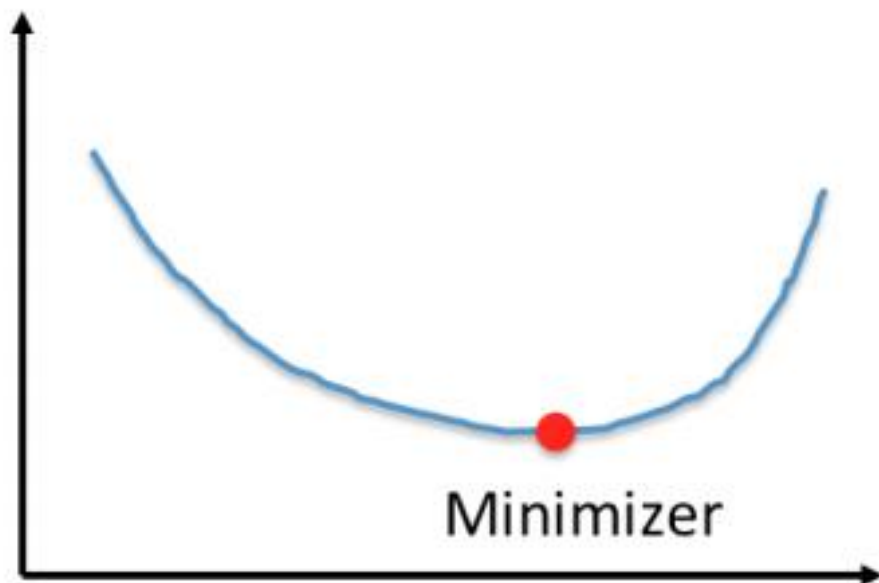
Two-feature problem fitted by plane

Convex vs Non-convex functions

$$\ell(w) = \sum_{i=1}^m (y_i - x_i^T w)^2$$

- Without going into formal definitions, a **convex** function is *generally bowl-shaped*. (see [examples](#) of convex functions)
- In linear regression, the squared loss is a convex function.
- The good thing about convex functions is that starting at any point, we can move towards the direction of lower value to get closer to the global minimum. With non-convex functions, we might be moving towards a *local* minima.

Convex



Non-Convex

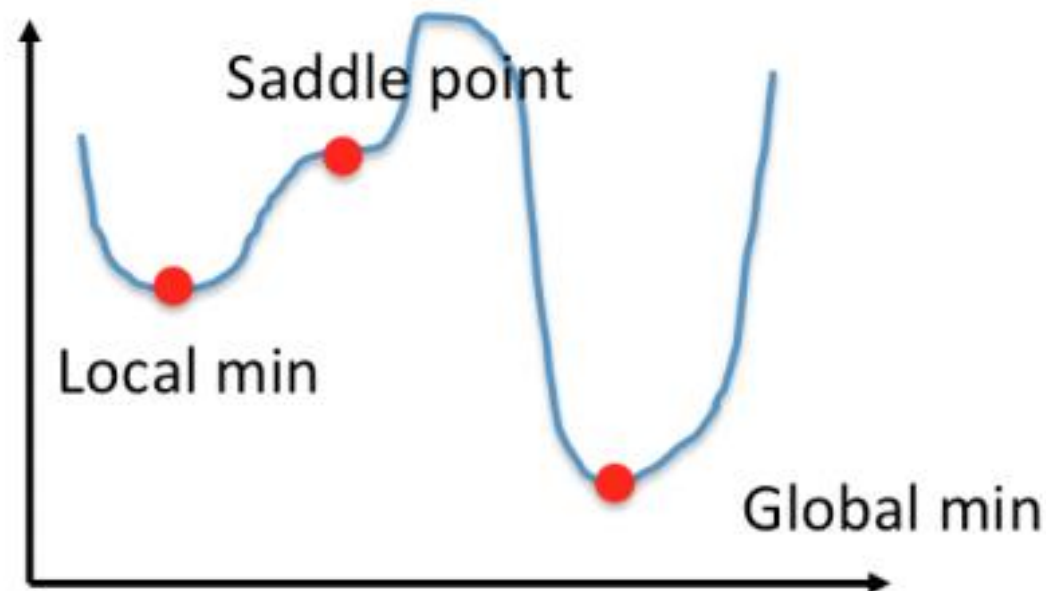


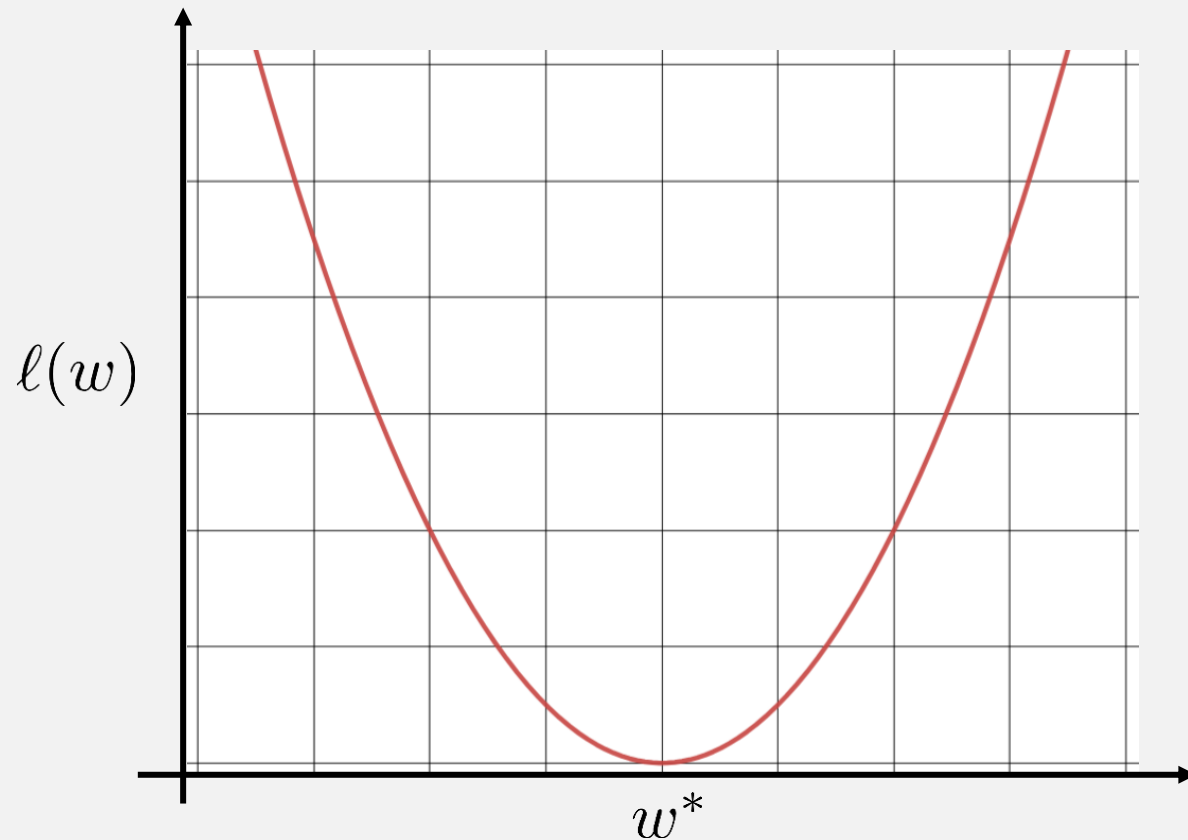
Image
from: [link](#)

Minimizing the Squared Loss

$$w^* = \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - x_i^T w)^2$$

$$\ell(w) = \sum_{i=1}^m (y_i - x_i^T w)^2$$

- In linear regression, the squared loss is a convex function. It looks *similar* to the below illustration, but in practice the exact bowl-shape will differ depending on the training data.



- As we vary the value of w , we get value for $\ell(w)$.
- We want the values for w that gives the minimum $\ell(w)$.
- How do we find w^* ?
- There are two approaches:
 - Closed-form solution
 - Gradient descent

Closed-form Solution

Function Minimization

- Consider a function $f(w)$ where we would like to find w^* that minimizes $f(w)$.
- This is an optimization problem, in particular a minimization problem.
- It is typically represented in the form below:

$$w^* = \arg \min_w f(w)$$

- Approaches to solve this varies depending on form of the function and parameter space. The field concerned with these problems is called optimization.
- For ML algorithms, we use gradient (derivative) based solutions.

Tools from calculus [1]

- Recall from calculus, one way to find the min of a function is to set the first derivative to 0 and solve.
- Let $g(w)$ be a function that takes a vector w as input and produces a scalar value $g : \mathbb{R}^{n+1} \rightarrow \mathbb{R}$. Then the gradient of g with respect to w is denoted as $\nabla_w g$ or $\frac{\partial g(w)}{\partial w}$. This is the vector of partial derivatives of g with respect to each element w_i of w .

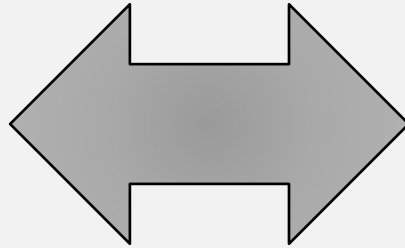
$$\nabla_w g = \frac{\partial g(w)}{\partial w} = \begin{bmatrix} \frac{\partial g(w)}{\partial w_0} \\ \frac{\partial g(w)}{\partial w_1} \\ \vdots \\ \frac{\partial g(w)}{\partial w_n} \end{bmatrix}$$

Tools from calculus [2]

- How do we find the minimum of a vector function? Recall from calculus, we set the gradient of the function to $\mathbf{0}$ and solve.

$$\nabla_w g = \mathbf{0}$$

Note that $\mathbf{0}$ is a vector of zeros



$$\begin{bmatrix} \frac{\partial g(w)}{\partial w_0} \\ \frac{\partial g(w)}{\partial w_1} \\ \vdots \\ \frac{\partial g(w)}{\partial w_n} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

- This is a system of equations that we solve for w , or more explicitly for the elements of w : $w_0, w_1, \dots, w_n$. The solution gives us the value of w that minimizes $g(w)$.

Minimizing a loss function [1]

- Take the partial derivative of the loss with respect to w :

$$\frac{\partial \ell(w)}{\partial w} = \frac{\partial}{\partial w} \sum_{i=1}^m (y_i - x_i^T w)^2 = \begin{bmatrix} \frac{\partial \sum_{i=1}^m (y_i - x_i^T w)^2}{\partial w_0} \\ \frac{\partial \sum_{i=1}^m (y_i - x_i^T w)^2}{\partial w_1} \\ \vdots \\ \frac{\partial \sum_{i=1}^m (y_i - x_i^T w)^2}{\partial w_n} \end{bmatrix}$$

Recall the chain rule for derivation:

$$\frac{\partial \sum_{i=1}^m (y_i - x_i^T w)^2}{\partial w_j} = -2 \sum_{i=1}^m (y_i - x_i^T w) x_{ij}$$

Minimizing a loss function [2]

$$\frac{\partial \ell(w)}{\partial w} = \begin{bmatrix} -2 \sum_{i=1}^m (y_i - x_i^T w) \\ -2 \sum_{i=1}^m (y_i - x_i^T w) x_{i2} \\ \vdots \\ -2 \sum_{i=1}^m (y_i - x_i^T w) x_{in} \end{bmatrix} = -2 \sum_{i=1}^m (y_i - x_i^T w) \begin{bmatrix} 1 \\ x_{i2} \\ \vdots \\ x_{in} \end{bmatrix} = -2 \sum_{i=1}^m (y_i - x_i^T w) x_i$$

$$\Rightarrow \frac{\partial \ell(w)}{\partial w} = -2 \sum_{i=1}^m (y_i - x_i^T w) x_i$$

Now we set to **0** and solve for **w**.

Minimizing a loss function [3]

- Set the partial derivative equal to **0** and solve for w :

$$\begin{aligned}\frac{\partial \ell(w)}{\partial w} &= -2 \sum_{i=1}^m (y_i - x_i^T w) \begin{bmatrix} 1 \\ x_{i2} \\ \vdots \\ x_{in} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \implies -2 \sum_{i=1}^m (y_i - x_i^T w) x_i = \mathbf{0} \\ &\implies \sum_{i=1}^m y_i x_i - \sum_{i=1}^m x_i^T w x_i = \mathbf{0} \\ &\implies \sum_{i=1}^m y_i x_i = \sum_{i=1}^m x_i^T w x_i\end{aligned}$$

Minimizing a loss function [4]

- Note that $\sum_{i=1}^m y_i x_i = X^T Y$. Also, $\sum_{i=1}^m x_i^T w x_i = X^T X w$. Thus:

$$\implies \sum_{i=1}^m y_i x_i = \sum_{i=1}^m x_i^T w x_i$$

$$\implies X^T Y = X^T X w \quad \text{multiply both sides by the inverse of } X^T X$$

$$\implies (X^T X)^{-1} X^T Y = (X^T X)^{-1} X^T X w$$

- Recall that for a square matrix A , then $AA^{-1} = A^{-1}A = I$. (Assuming the inverse exists)

$$\implies w = (X^T X)^{-1} X^T Y$$

This is the w that minimizes the loss function

Another derivation using vector-matrix calculus

- Recall, the goal:

$$w^* = \arg \min_{w \in \mathbb{R}^{n+1}} \ell(w)$$

$$= \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - x_i^T w)^2$$

$$= \arg \min_{w \in \mathbb{R}^{n+1}} \|Y - Xw\|_2^2$$

$$= \arg \min_{w \in \mathbb{R}^{n+1}} (Y - Xw)^T (Y - Xw) \longrightarrow \text{We'll use this formula}$$

Another derivation – Useful rules from calculus

- If a and x are vectors, then:

$$\frac{\partial(a^T x)}{\partial x} = \frac{\partial(x^T a)}{\partial x} = a$$

- If A is a symmetric matrix and x is a vector, then:

$$\frac{\partial(x^T A x)}{\partial x} = 2Ax$$

Another derivation [1]

$$\begin{aligned}\ell(w) &= (Y - Xw)^T(Y - Xw) \\ &= (Y^T - w^T X^T)(Y - Xw) \\ &= Y^T Y - Y^T Xw - w^T X^T Y + w^T X^T Xw \\ &= Y^T Y - 2w^T X^T Y + w^T X^T Xw\end{aligned}$$

since $(Xw)^T = w^T X^T$

expanding the parantheses

since $(w^T X^T)Y = Y^T(Xw)$

$$\begin{aligned}\frac{\partial \ell(w)}{\partial w} &= \frac{\partial(Y^T Y - 2w^T X^T Y + w^T X^T Xw)}{\partial w} \\ &= \frac{\partial(Y^T Y)}{\partial w} - 2\frac{\partial(w^T X^T Y)}{\partial w} + \frac{\partial(w^T X^T Xw)}{\partial w}\end{aligned}$$

$$= \mathbf{0} - 2X^T Y + 2X^T Xw$$

using rules from previous slide

Another derivation [2]

$$\frac{\partial \ell(w)}{\partial w} = -2X^T Y + 2X^T X w$$

- We set partial derivative to **0** and solve for w .

$$-2X^T Y + 2X^T X w = 0$$

$$\implies X^T X w = X^T Y \quad \text{multiply both sides by the inverse of } X^T X$$

$$\implies (X^T X)^{-1} X^T X w = (X^T X)^{-1} X^T Y$$

$$\implies w = (X^T X)^{-1} X^T Y$$

Closed form solution

- Recall, the goal: $w^* = \arg \min_{w \in \mathbb{R}^{n+1}} \ell(w)$

$$= \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - x_i^T w)^2$$

$$= \arg \min_{w \in \mathbb{R}^{n+1}} \|Y - Xw\|_2^2$$

$$= \arg \min_{w \in \mathbb{R}^{n+1}} (Y - Xw)^T (Y - Xw)$$

- $X^T X w = X^T Y$ is known as the normal equation.
- Assuming $X^T X$ is invertible, then the closed form solution is given by:

$$w^* = (X^T X)^{-1} X^T Y$$

Closed form solution in python using numpy

```
1 # Note that matrix multiplication operator in numpy is @
2 # So, matrix A multiplied by matrix B is A@B
3
4 XtX_inverse = np.linalg.inv(X.T @ X) # compute inverse of X.T @ X
5
6 w_closed_form_solution = XtX_inverse @ (X.T @ Y)
```

LinearRegression in scikit-learn

```
class sklearn.linear_model.LinearRegression(*, fit_intercept=True,  
copy_X=True, n_jobs=None, positive=False) # \[source\]
```

```
from sklearn.linear_model import LinearRegression  
  
lr_model = LinearRegression()  
lr_model.fit(X_train, y_train)  
  
# get learnt feature weights  
lr_model.coef_  
# get learnt bias/intercept  
lr_model.intercept_  
# predict  
lr_model.predict(X_test)
```

- Scikit-learn's [LinearRegression](#) computes the closed-form solution (does not use gradient descent).
- Note: scikit-learn by default assumes you did not add 1 column to your data. See the documentation of *fit_intercept* argument.

What if $X^T X$ is not invertible?

- This is an involved topic in linear algebra. Here we'll give simplified overview.
- When $X^T X$ is invertible, there is a single unique squared-loss minimizer w^* .
- $X^T X$ being not invertible typically occurs when we have more features than instances in our training set (wide data matrix).
- In this case there are many possible w^* that achieve minimum squared-loss. What we can do is then get the w^* that has the smallest 2-norm $\|w^*\|_2$.
- We can do this using the Moore-Penrose pseudoinverse denoted as X^+ . $w^* = X^+ Y$
- Scikit-learn LinearRegression finds the squared-loss minimizers with minimum 2-norm.
- You can also use numpy's [pinv](#) to get the pseudoinverse.
- Alternatively, we can apply L2-regularization to linear regression which has a unique solution (covered later in this lecture).
- Note: when $X^T X$ is invertible, $X^+ = (X^T X)^{-1} X^T$.

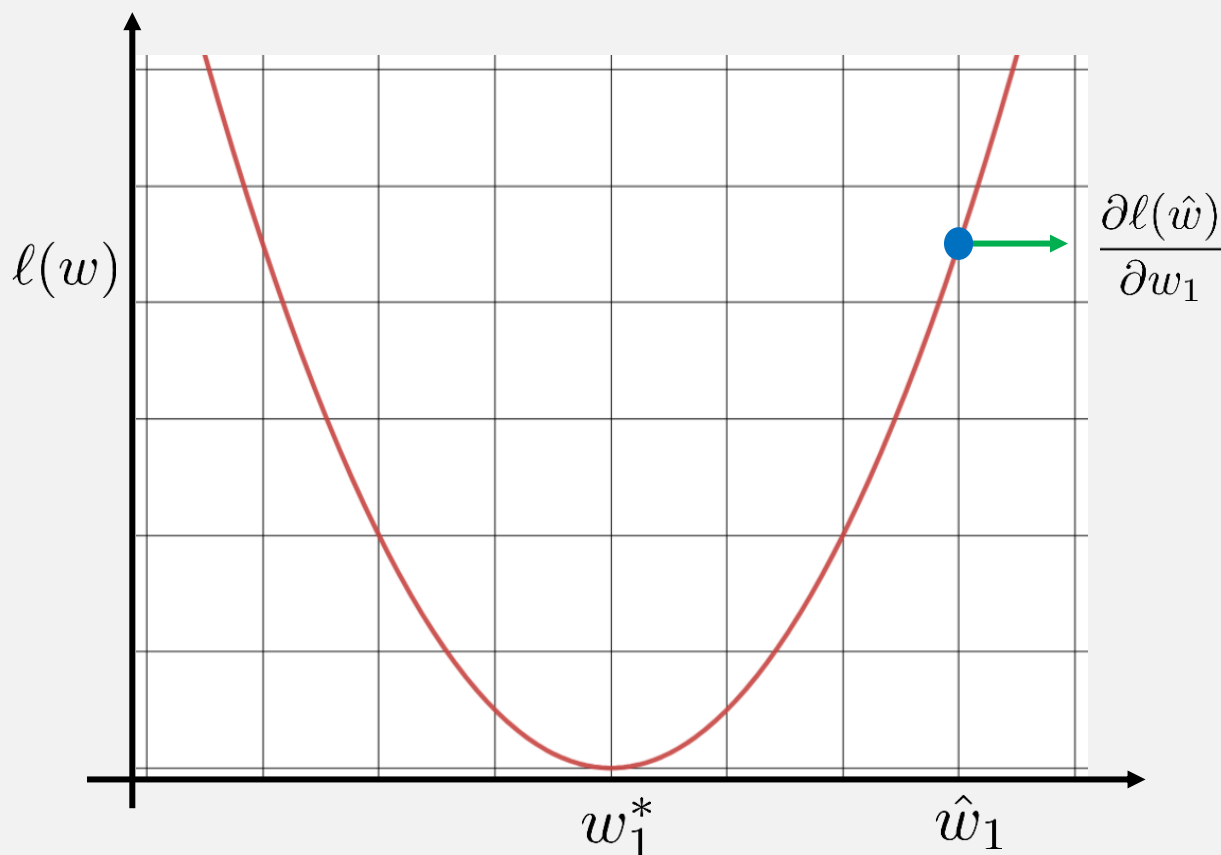
Gradient Descent Solution

Gradient Descent Overview

- The closed form solution requires inverting $X^T X$ which can be computationally demanding if $X^T X$ is large.
- Gradient descent is an iterative technique for finding the minimum of a function.
- Recall earlier in the slides, that we denote the gradient as ∇_w .
- The gradient of a function g at a vector a , denoted as $\nabla g(a)$, is a vector. This vector gives us the **direction** and **magnitude** of the **steepest (largest) increase** for the function g at the point a .

Gradient Descent – Visualization with a single feature **without** intercept [1]

- $\nabla \ell(w)$ is a vector defining the **direction** and **magnitude** of the steepest increase for the function ℓ at the point w .



- Consider a problem where we have **one** feature and we consider linear hypotheses **without** intercept, so w only has one element:

$$\mathcal{H} = \{h(x; w_1) = x_1 w_1 \mid w_1 \in \mathbb{R}\}$$

- We want the values for w_1 that gives the minimum loss. We denote this as w_1^* .
- Suppose we randomly initialize our weight $\hat{w}_1$. Its loss is the blue point.
- We can either adjust $\hat{w}_1$ by going left or right on the weight axis. We want to go left so that we decrease the loss and get closer to w_1^* .
- The green vector shows the direction of the largest increase of the loss curve $\ell(w)$ at $\hat{w}_1$ along the components of w . This vector tells us how to
- The green vector is the gradient giving the **direction** and **magnitude** of **steepest increase** for the loss curve $\ell(w)$ at $\hat{w}_1$.
- Importantly, $\frac{\partial \ell(\hat{w}_1)}{\partial w_1}$ tells us which direction to push $\hat{w}_1$ in order to increase the loss value $\ell(w)$.

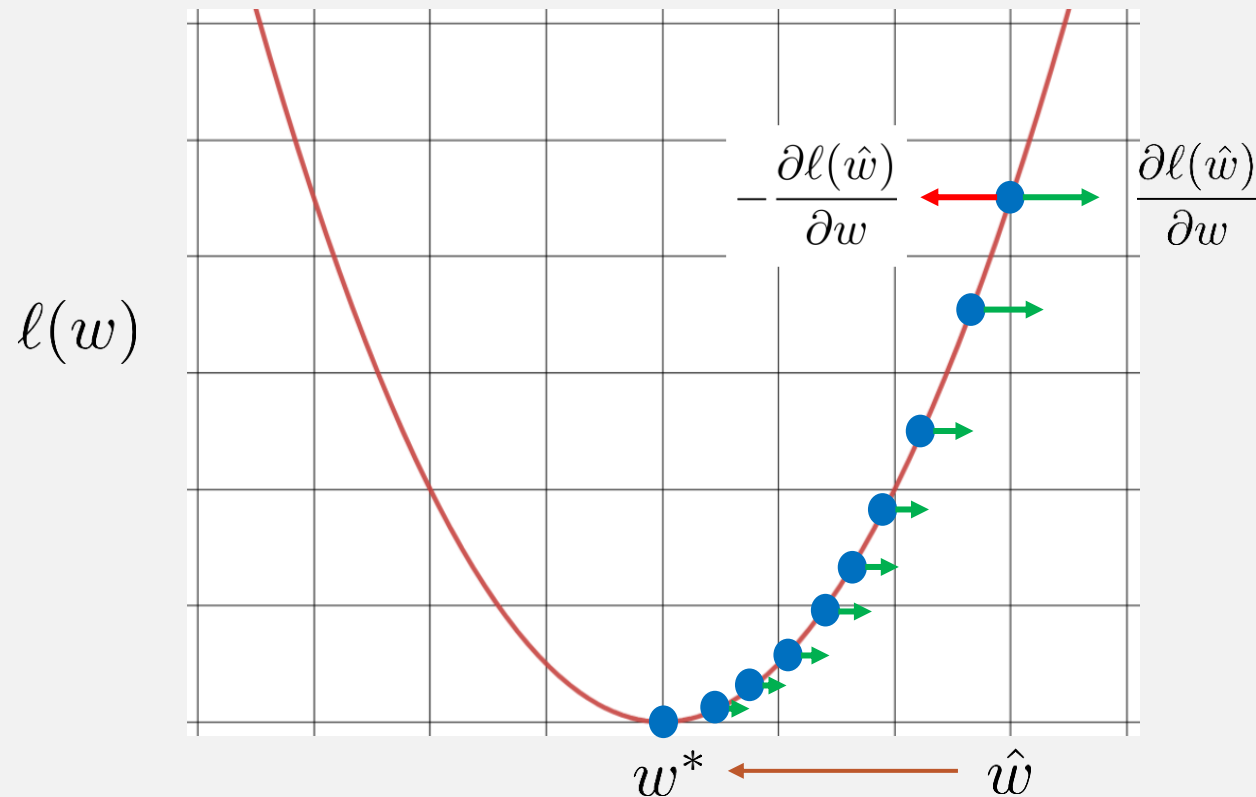
Gradient Descent – Visualization with a single feature **without** intercept [2]

- Basic idea: take a step in the **opposite** direction of the gradient. This will get you closer to the minimum loss.

$$\nabla_w \ell(\hat{w}) = 2X^T(X\hat{w} - Y)$$

$$\hat{w} := \hat{w} - \alpha \nabla_w \ell(\hat{w})$$

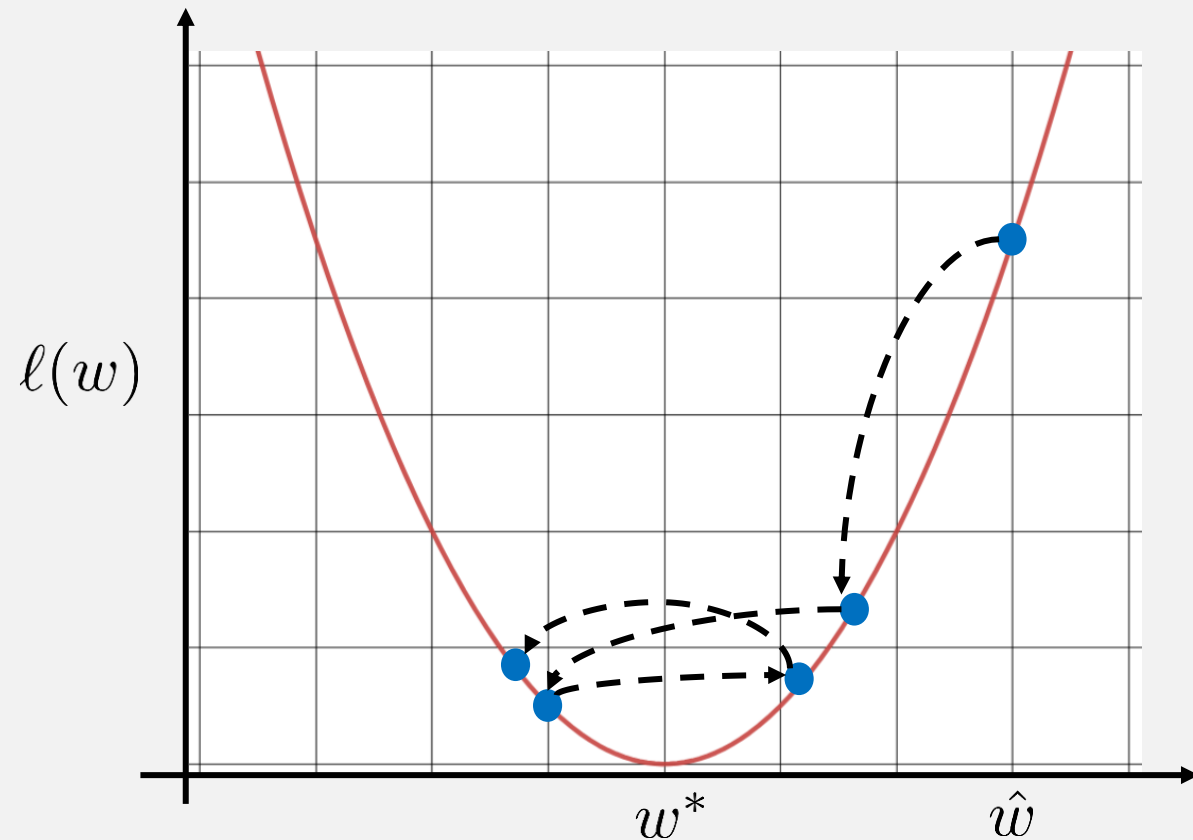
$$:= \hat{w} - \alpha X^T(X\hat{w} - Y)$$



- To go in the **opposite** direction of the gradient, we just negate the gradient.
- α is called the **learning rate**. It scales the gradient value. Typically set between 0 and 1.
- We iteratively apply this update to reduce the loss.
- We iterate until a defined number of iterations. Or until the loss stops decreasing, so we have reached the global minimum. Recall the gradient at the global minimum will be the zero vector.

Effect of α – the learning rate [1]

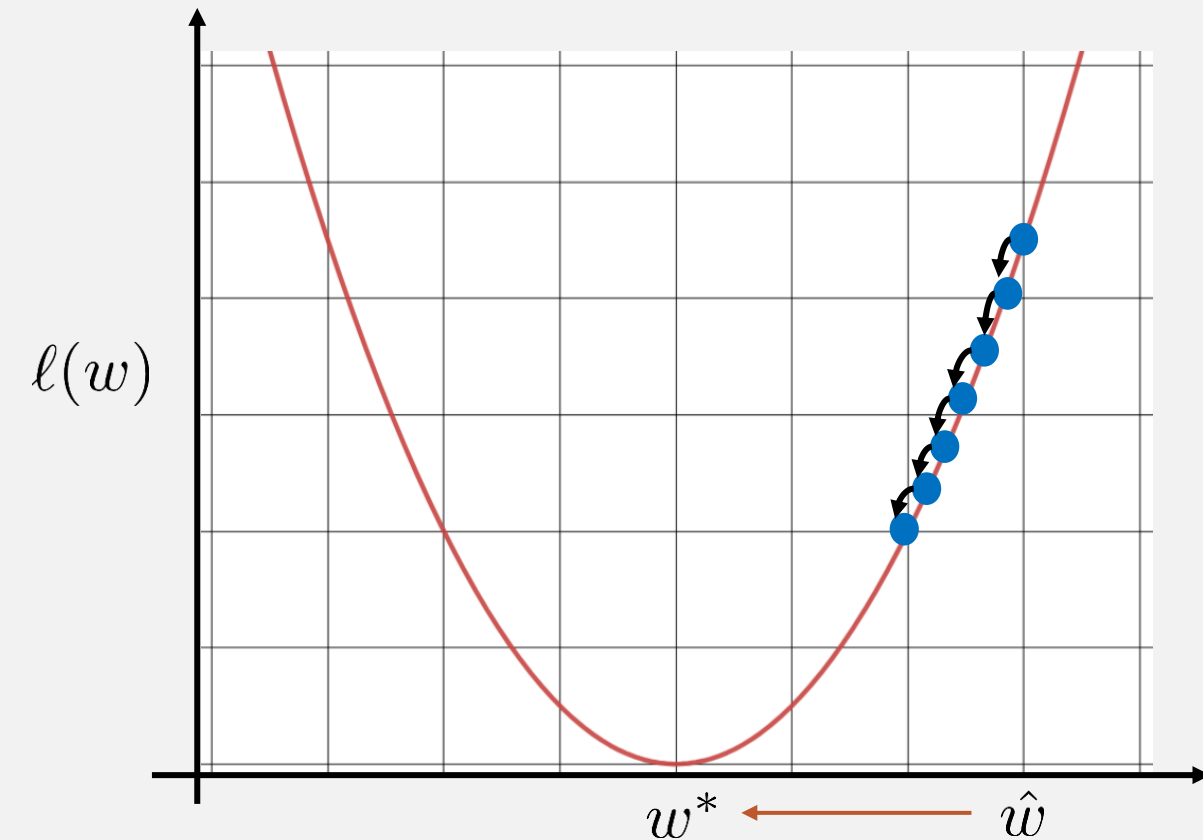
- The learning rate α typically set between 0 and 1.



- α is called the learning rate. It scales the gradient value.
- Making this too large, then we descend towards the minimum faster, but we might overshoot the minimum.
- As illustrated in plot, in such a case, it is stuck and unable to get close to w^* .

Effect of α – the learning rate [2]

- The learning rate α typically set between 0 and 1.



- Making this too small and we might take a lot of iterations to reach the minimum.
- α can be set to a larger value at the start, but then reduced after a number of iterations. This is known as scheduling the learning rate.
- For example, at the start we set it to 0.1, but then after 100 iterations we set it to 0.01.
- There are more advanced techniques for scheduling that we won't cover (momentum, RMSprop, adam, adagrad)

Effect of α – the learning rate [3]

- Typical value for α is 0.01 or 0.1. However, note that it really depends on your dataset and problem. Too high α can lead to training loss increasing.

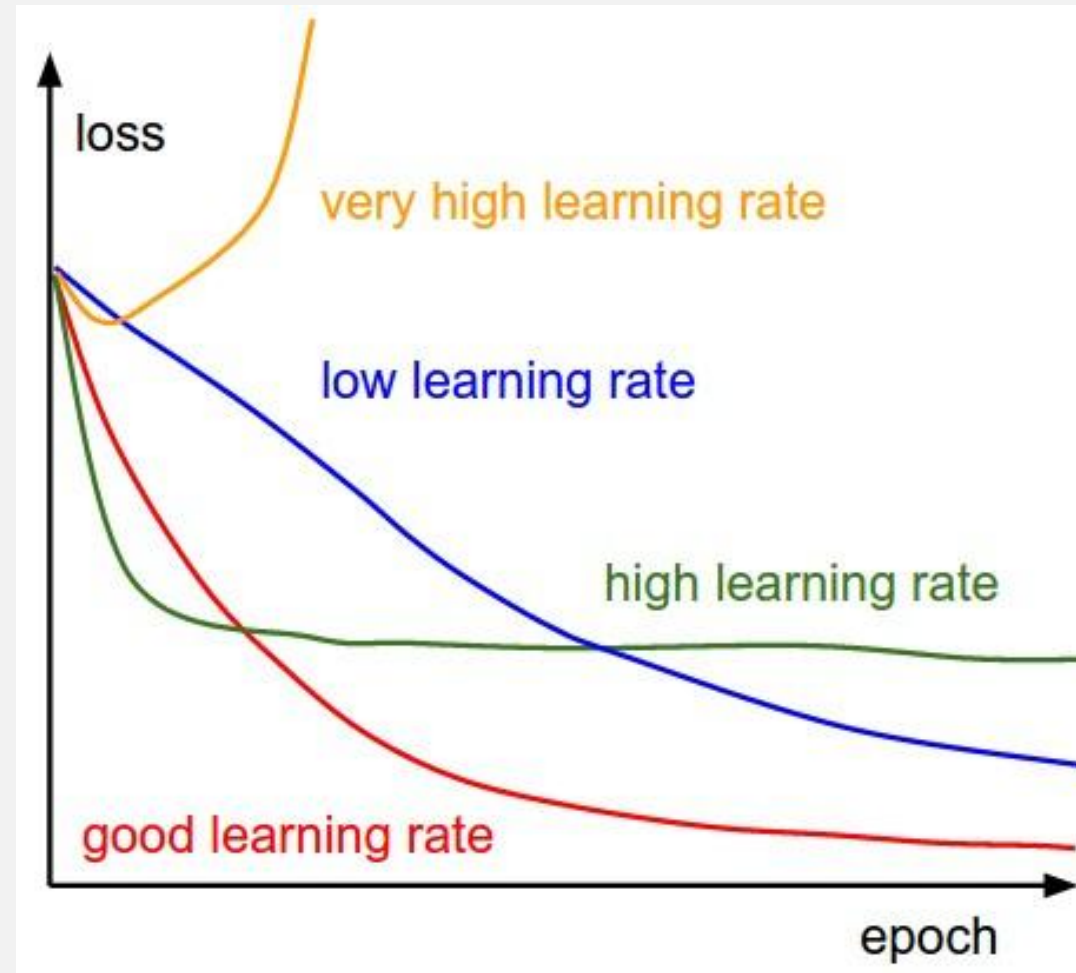
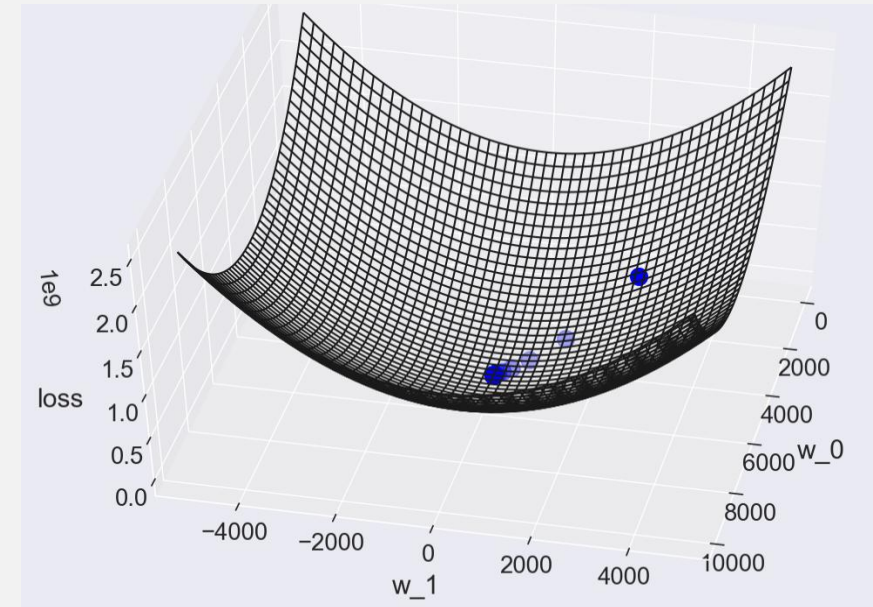
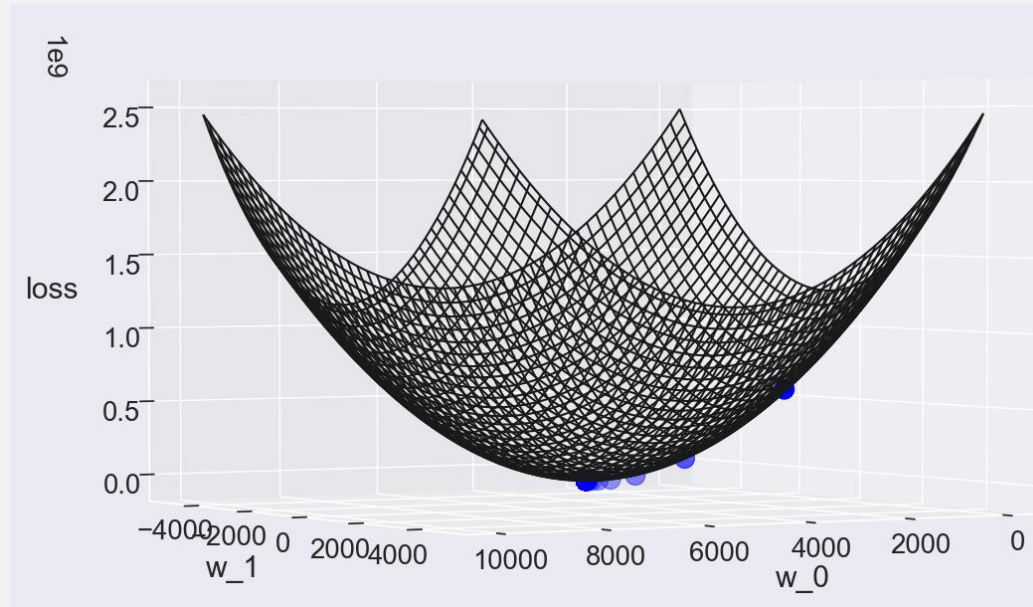


Image from: [link](#)

Gradient Descent – Visualization with a single feature **with** intercept [1]

- Now consider a problem with one feature and we consider hypotheses with intercept. So, $w \in \mathbb{R}^2$ has two elements.

$$\nabla_w \ell(w) = \begin{bmatrix} \frac{\partial \ell(w)}{\partial w_0} \\ \frac{\partial \ell(w)}{\partial w_1} \end{bmatrix}$$



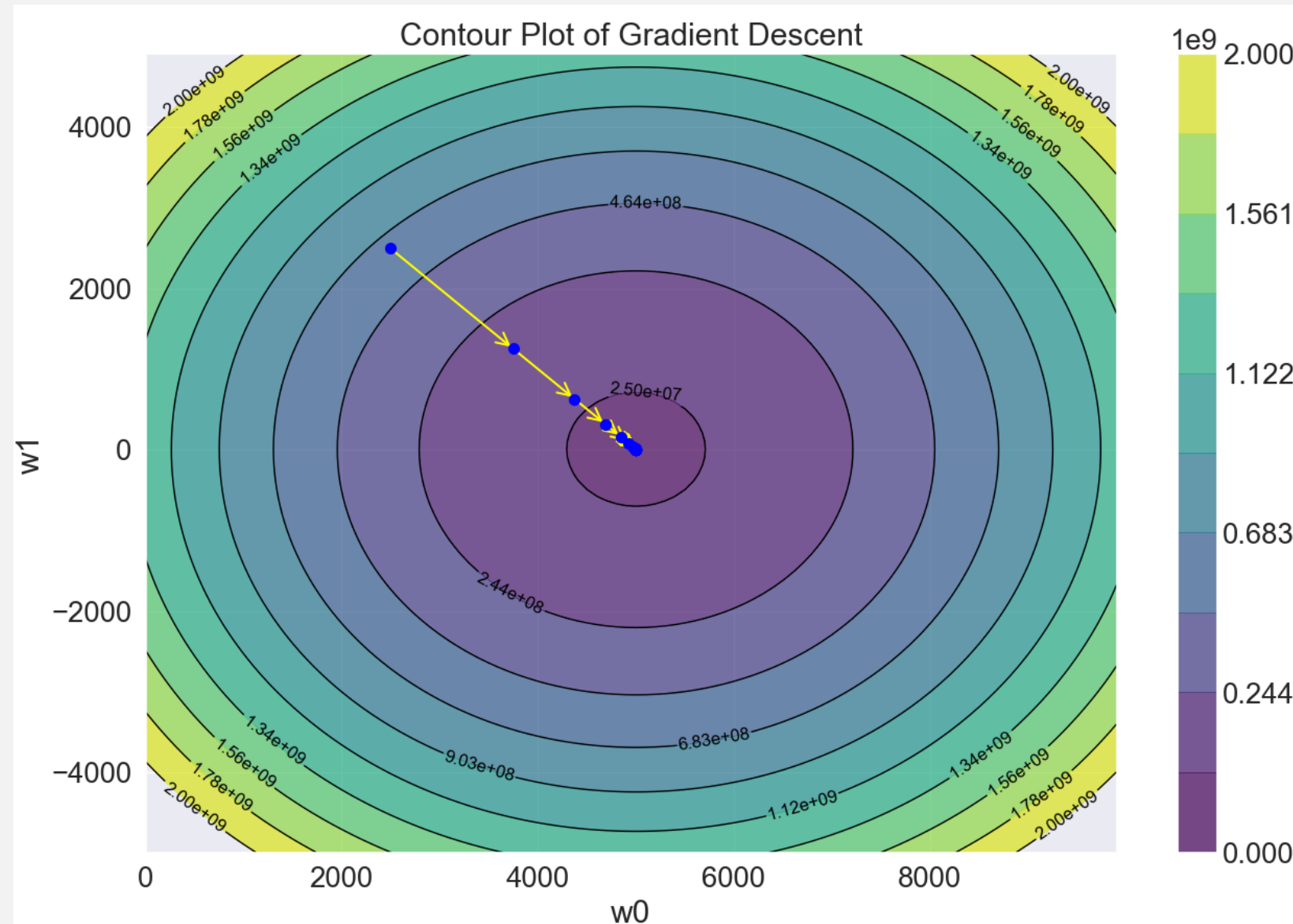
- In this case, the loss curve is in 3D with two axes for each element of w . But it is still convex with a global minimum. The above two images show gradient descent with the blue dots from two perspectives for same problem.

See this video on 3D gradient vector visualization: [link](#)

Gradient Descent – Visualization with a single feature **with** intercept [2]

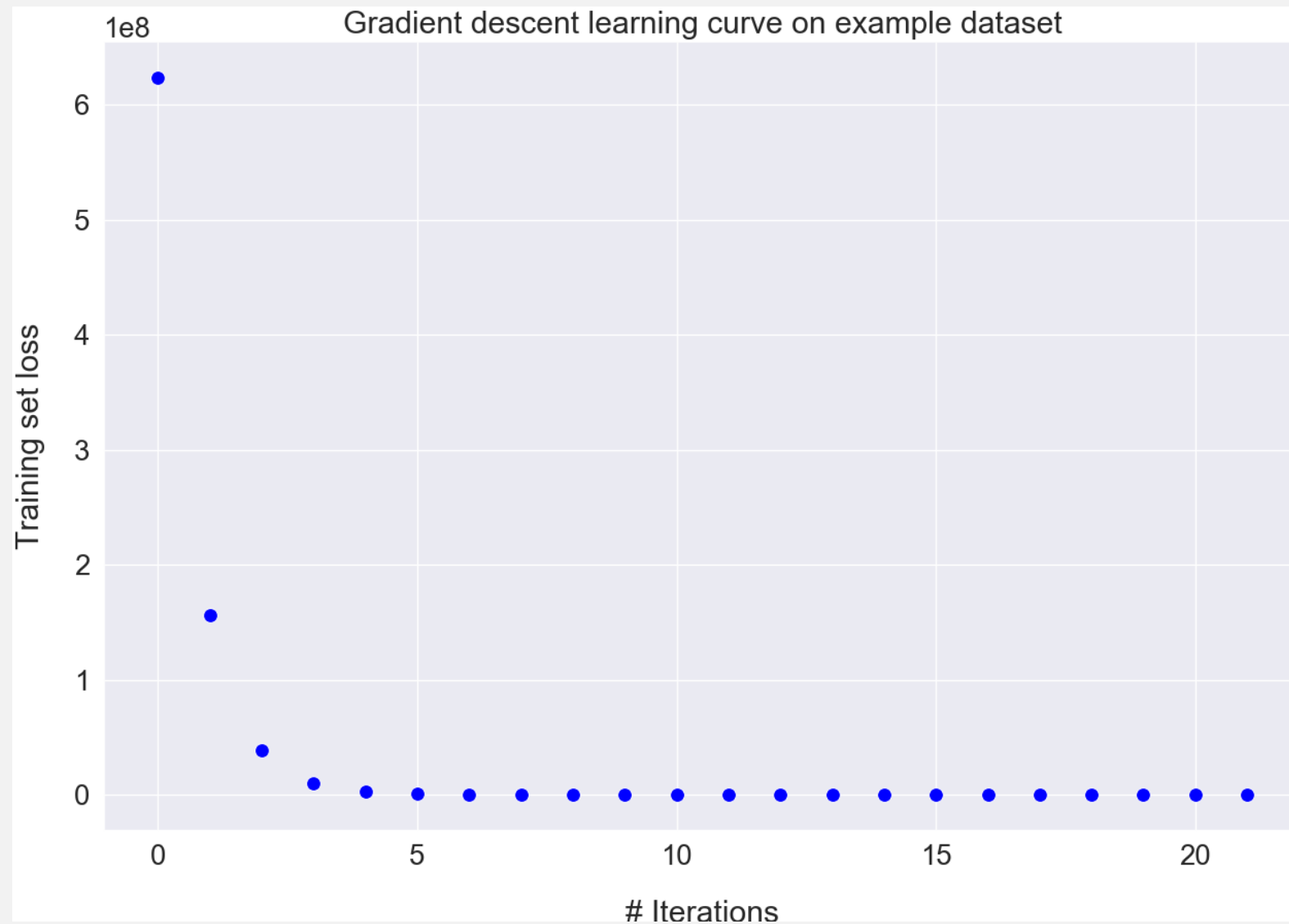
- We can also use [contour plots](#) to visualize the gradient descent updates.
- Yellow arrows indicate the gradient vector at each iteration:

$$\nabla_w \ell(w) = \begin{bmatrix} \frac{\partial \ell(w)}{\partial w_0} \\ \frac{\partial \ell(w)}{\partial w_1} \end{bmatrix}$$



- Learning curve: plot a dataset's loss over iterations of GD.

$$\frac{1}{m} \sum_{i=1}^m (y_i - x_i^T \hat{w})^2$$



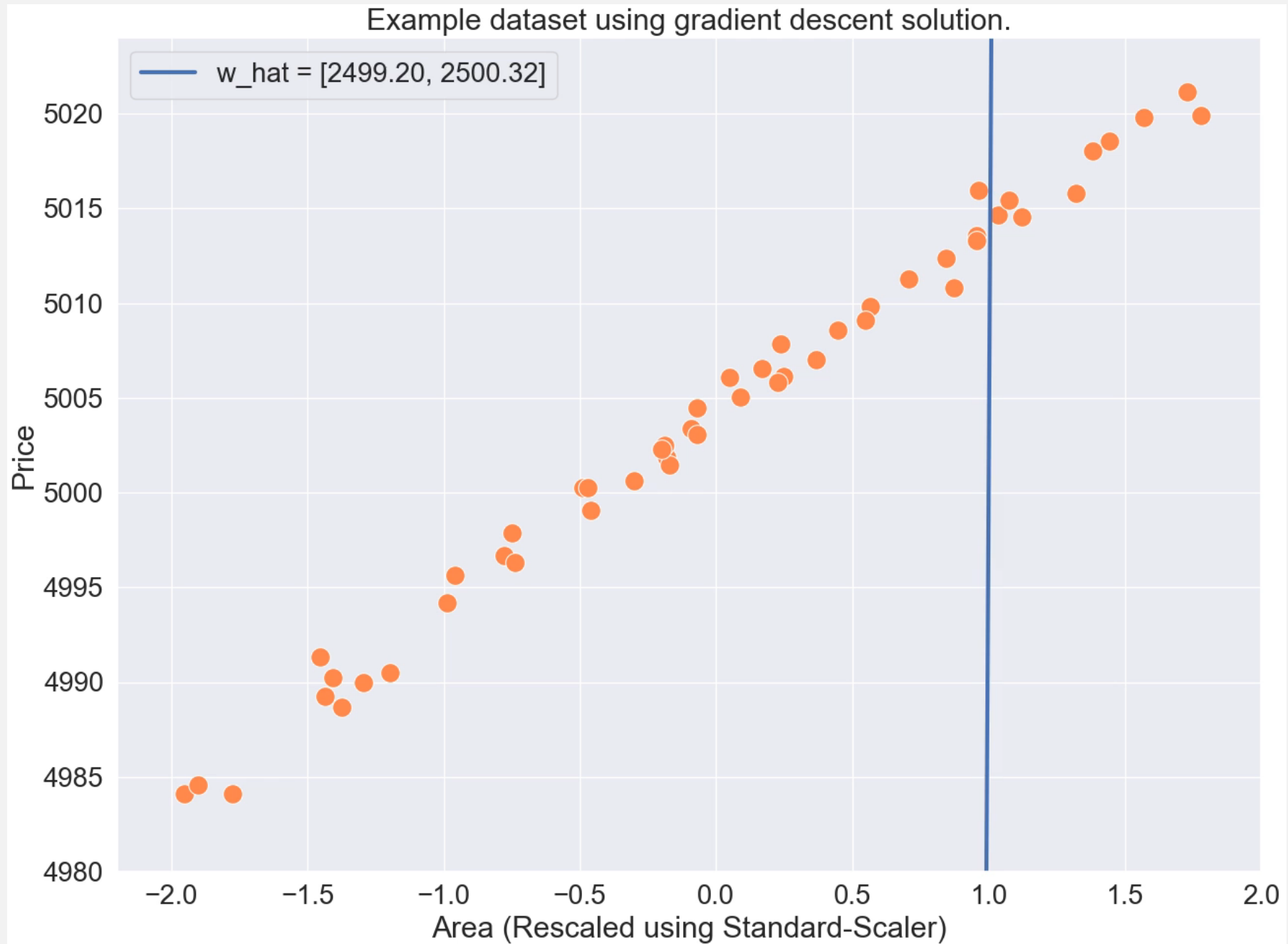
Summary of gradient descent solution to linear regression

- Recall, the goal: $w^* = \arg \min_{w \in \mathbb{R}^{n+1}} \ell(w)$
$$= \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - x_i^T w)^2$$
$$= \arg \min_{w \in \mathbb{R}^{n+1}} \|Y - Xw\|_2^2$$
$$= \arg \min_{w \in \mathbb{R}^{n+1}} (Y - Xw)^T (Y - Xw)$$
- Randomly initialize $\hat{w}$. Set $\alpha \in (0, 1)$. For N iterations or until the loss stops decreasing, perform the following update:

$$\begin{aligned}\hat{w} &:= \hat{w} - \alpha \nabla_w \ell(\hat{w}) \\ &:= \hat{w} - \alpha X^T (X\hat{w} - Y)\end{aligned}$$

- Then we can predict $\hat{y}_{new} = x_{new}^T \hat{w}$

Linear Regression GD Animation



Gradient descent in python using numpy and loops

```
1 start_time = time.time()
2
3 np.random.seed(1000)
4 w_hat = np.random.normal(loc=0.0, scale=1.0, size=(X.shape[1],1)).round(2) # randomly initialize weight vector
5
6 mse_losses = [] # maintain each iteration loss
7
8 alpha = 1e-2 # set alpha
9 eps = 1e-3 # set stopping criteria: if my loss stops changing by this amount, then stop
10
11 for i in range(1000): # loop for 1000 iterations, we can increase this if needed
12
13     mse_loss_i = np.sum((output - (X@w_hat))**2) # compute current w_hat loss
14     mse_losses.append(mse_loss_i)
15
16     gradient_loss = X.T@(X@w_hat - output) # current iteration gradient
17     w_hat = w_hat - alpha*gradient_loss # update w_hat using alpha*gradient
18
19     if i > 1 and np.abs(mse_losses[-1] - mse_losses[-2]) <= eps: # if loss stops changing by a tolerance, then stop
20         break
21
22 mse_losses = np.array(mse_losses) # can plot the loss at each iteration
23
24 gd_time = time.time()-start_time
25 print('# updates: {}. time: {} millisec'.format(i, gd_time))
```

Mini-batch Gradient Descent

Mini-batch Gradient Descent [1]

- The closed form solution requires inverting $X^T X$ which can be computationally demanding if $X^T X$ is large.
- The gradient descent solution involves the following update:

$$\hat{w} := \hat{w} - \alpha X^T (X \hat{w} - Y)$$

- The size of the data matrix X maybe too large to fit in RAM. Or we may want to make use of GPU devices to speed up computation, but the VRAM of the GPU device is limited.
- Mini-batch gradient descent is a minor modification to gradient descent that also finds the global minimum w^* .

Mini-batch Gradient Descent [2]

- Randomly initialize $\hat{w}$. Set $\alpha \in (0, 1)$. For N iterations or until the loss stops decreasing, perform the following:
 1. Randomly permute (shuffle) your data (X, Y) .
 2. Split your shuffled data into k batches $(X_1, Y_1), (X_2, Y_2), \dots, (X_k, Y_k)$
 3. For each of your k batches (X_i, Y_i) , perform the update:

$$\hat{w} := \hat{w} - \alpha X_i^T (X_i \hat{w} - Y_i)$$

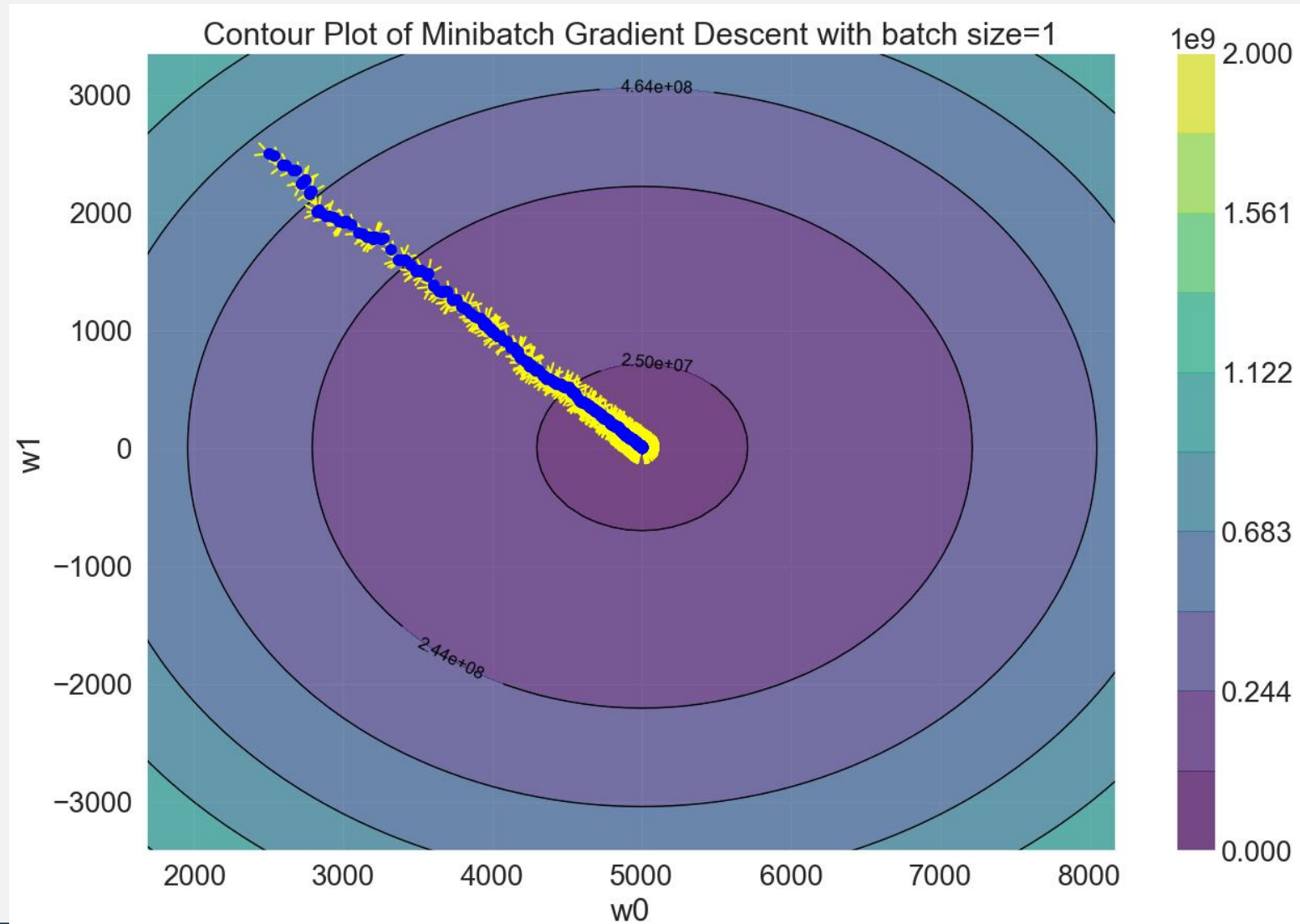
- Note that steps 1, 2, and 3 are done on each of the N iterations. So, the number of updates to $\hat{w}$ is Nk .
- The loss is computed on the batches and then averaged. This average batch loss is an approximation for the average loss on X .

Mini-batch Gradient Descent [3]

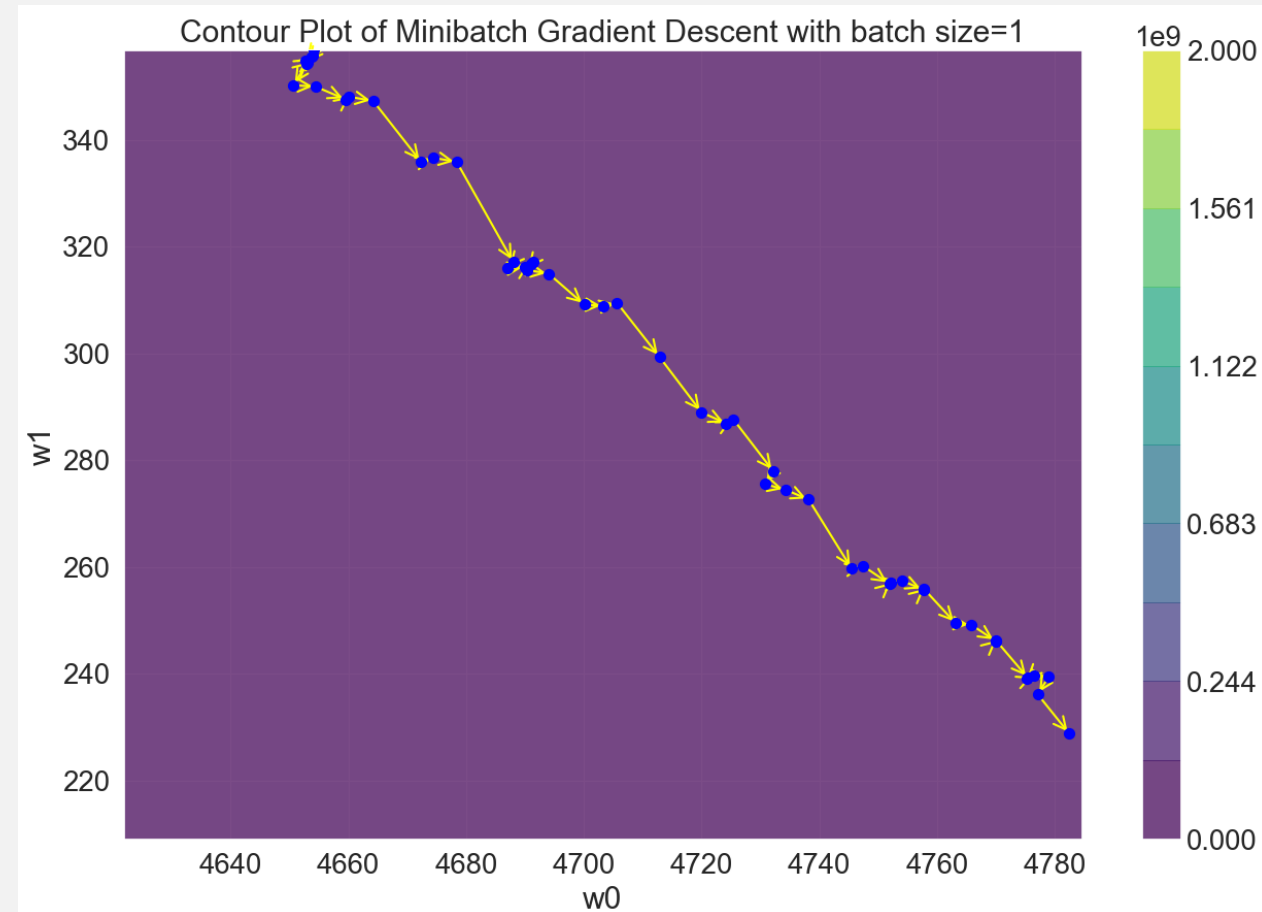
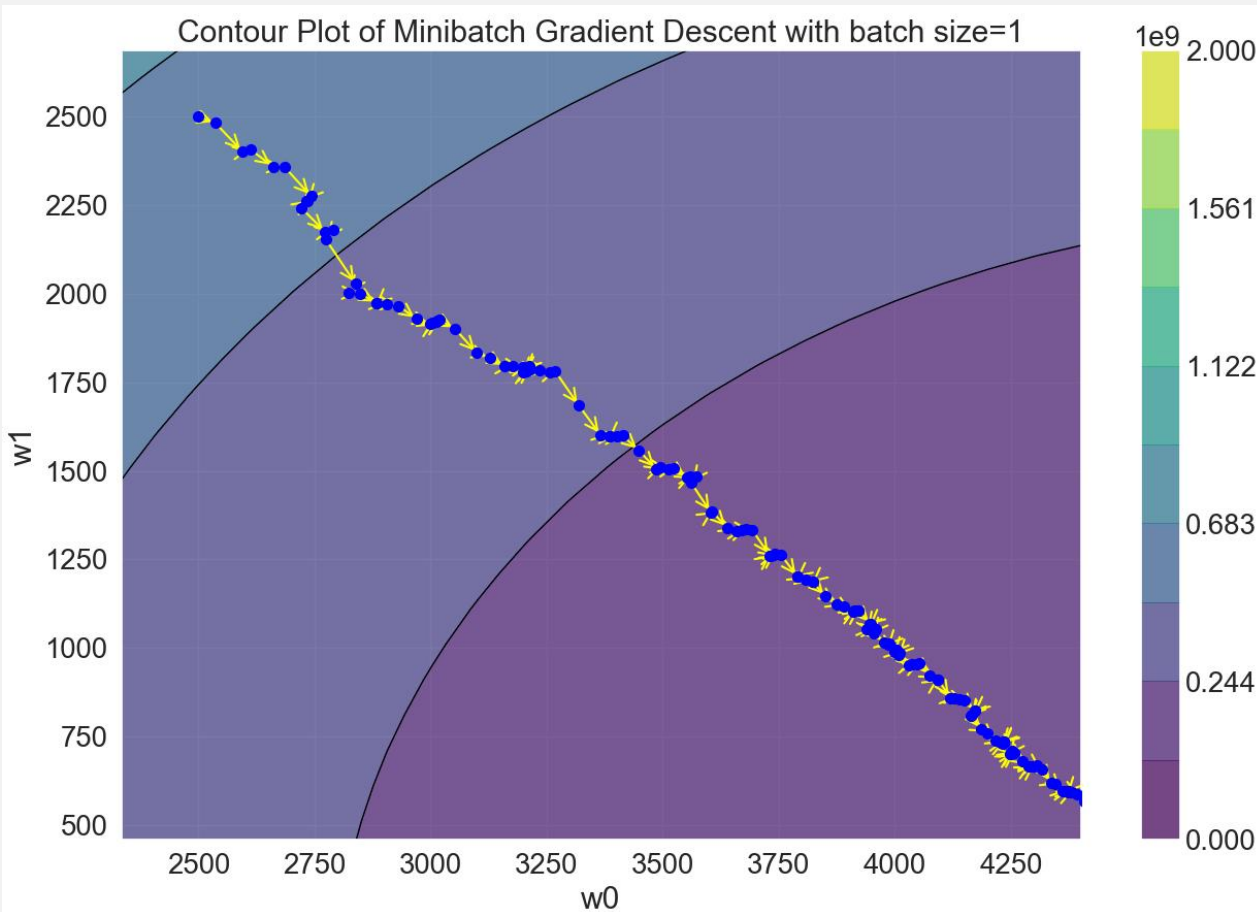
- The reason for random shuffling is to remove ordering and for statistical purposes. We are mimicing sampling a dataset (batch) from the underlying distribution (full dataset) without replacement.
- Stochastic gradient descent (SGD): we set the batch size to 1 instance.
- Full-batch or batch GD: we set the batch size to the entire dataset.
- Mini-batch GD: we set the batch size between 2 and entire dataset.
- However, the term SGD is used by some to refer to any form of batched gradient descent regardless of batch size.

Mini-batch Gradient Descent – Visualization with a single feature **with** intercept [1]

- Contour plot of mini-batch GD with batch size of 1. Notice the noisy path.

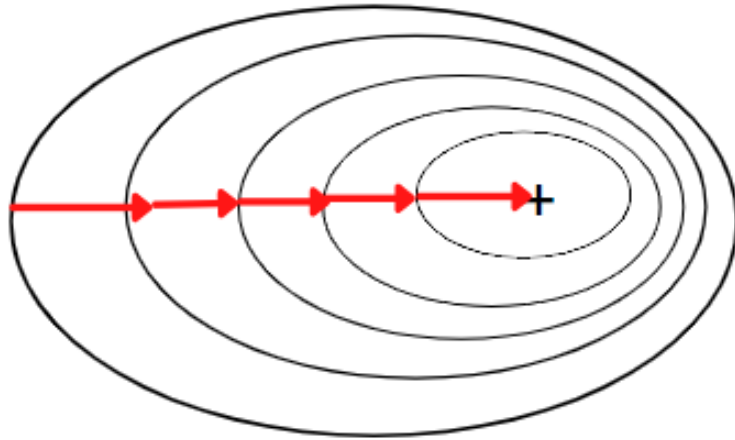


Mini-batch Gradient Descent – Visualization with a single feature **with** intercept [2]

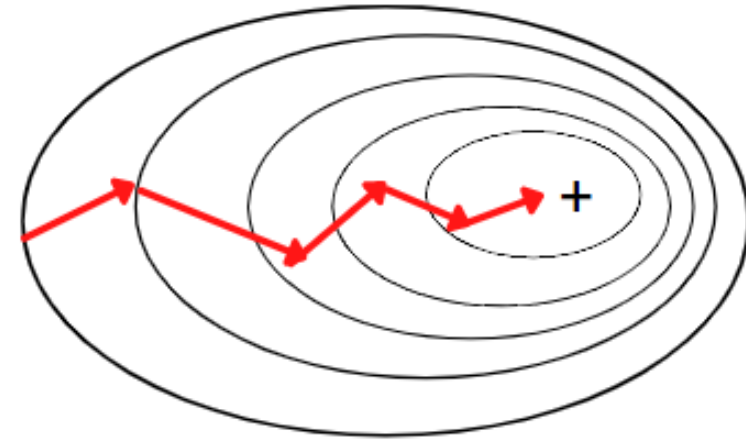


General Visualization of full-batch GD vs Mini-batch GD vs Stochastic GD (batch_size=1)

Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent

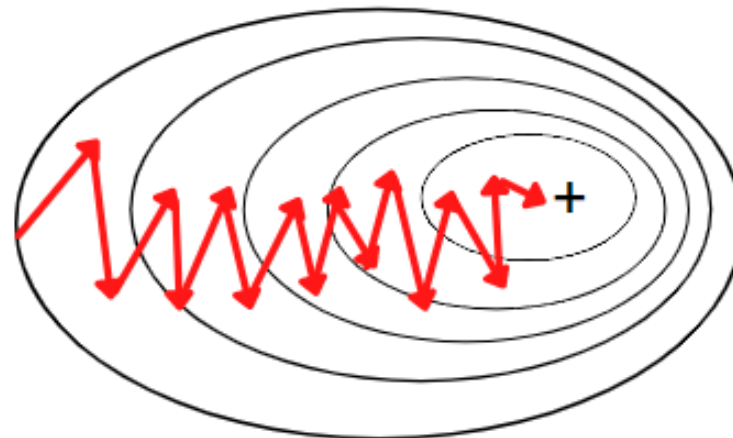


Image from: [link](#)

Mini-batch Gradient Descent Remarks

- From experimenting, researchers noticed that mini-batch GD can reach the minimum in fewer *iterations* than full-batch gradient descent. Each iteration in mini-batch GD does several weight updates, but computational cost is smaller than in full-batch GD.
- For linear regression, mini-batch GD is an alternative when our data is too large to fit into RAM or too costly to perform matrix multiplication. In which case, batching can help alleviate these issues.
- In neural networks, the loss is almost always non-convex. The noise in mini-batch GD can help escape local minimas which are encountered. This is not an issue in linear regression since the loss is convex.
- The time-savings in using mini-batch GD is more pronounced in neural networks where we have more complex computations.

```

batch_size = 128
n_instances = X_train.shape[0]
iter_losses = []
for iter_i in range(n_iters):
    # randomly shuffle the training set
    random_indices = np.random.permute(n_instances)

    batch_losses = []
    for batch_i in range(n_instances // batch_size):
        batch_indices = random_indices[batch_i*batch_size: (batch_i+1)*batch_size]
        X_batch, y_batch = X_train[batch_indices,:], y_train[batch_indices,:]

        # do mini-batch weight updates
        ...

        # compute batch_loss and add to batch_losses
        ...
        batch_losses.append(current_batch_loss)

    # compute average batch_loss and add to iter_losses
    iter_losses.append(np.mean(batch_losses))

```

SGDRegressor in scikit-learn

```
from sklearn.linear_model import SGDRegressor

lr_model = SGDRegressor(max_iter=1000)
lr_model.fit(X_train, y_train)

# get learnt feature weights
lr_model.coef_
# get learnt bias/intercept
lr_model.intercept_
# predict
lr_model.predict(X_test)
```

- Recall that scikit-learn's [LinearRegression](#) computes the closed-form solution (does not use gradient descent).
- [SGDRegressor](#) does SGD (batch size of 1). Also, has [partial_fit](#)(X_batch, y_batch) for you to use your own batch updates.
- Note that the learning rate and max iterations are examples of *hyperparameters* since you set them before training. The linear regression weights are *parameters* of the algorithm.

Stability of gradient descent and feature scaling

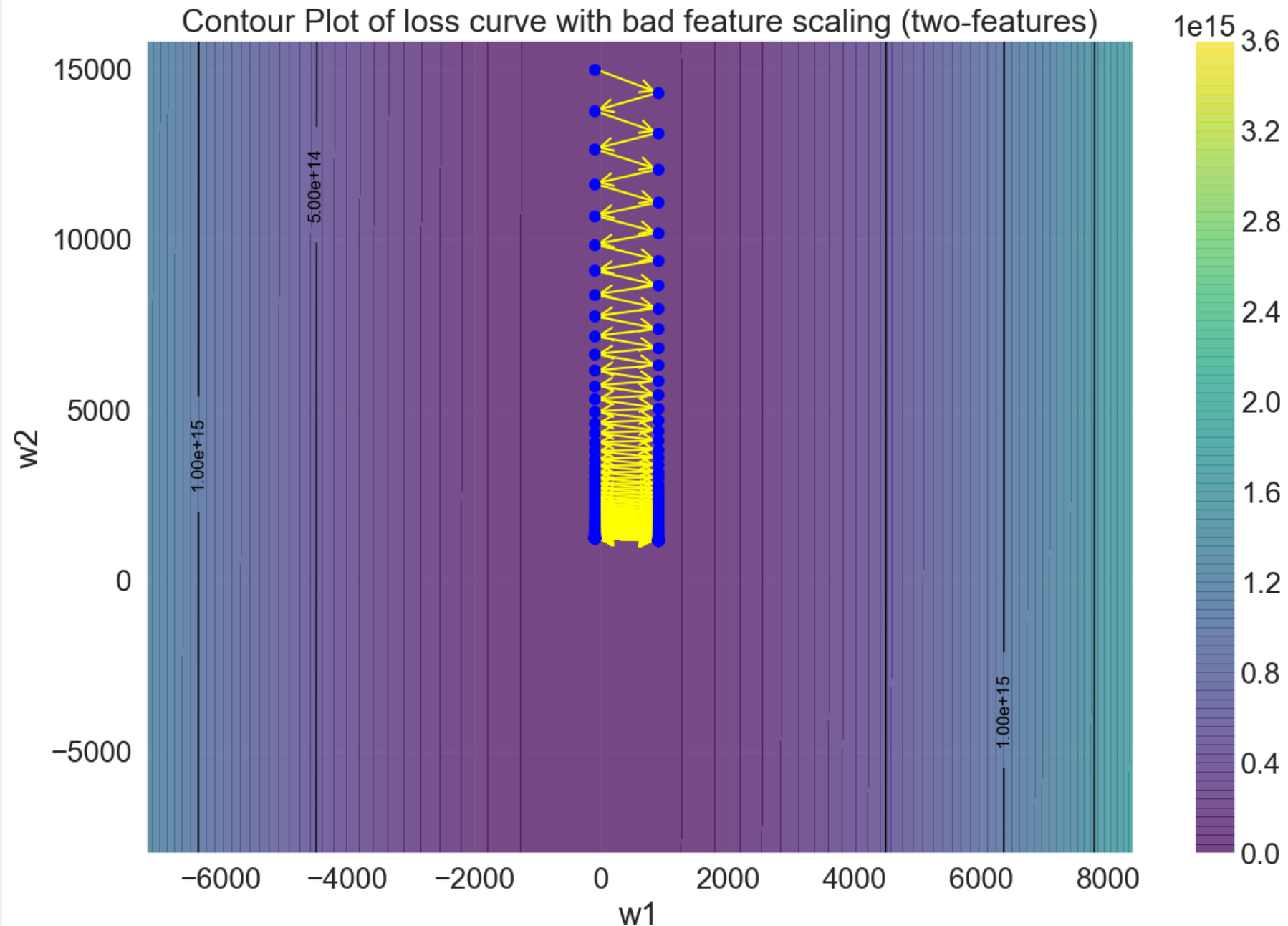
Feature scaling in GD

- If one feature has a much larger range and spread (standard deviation) than the others, then this feature will skew the loss curve.
- Thus, when doing gradient descent, it will require more iterations to reach the global minimum.
- In some extreme skew case, you may not reach the global minimum unless you reduce the learning rate significantly.
- Therefore, you may want to apply `StandardNormalScaler` to the features of your data before applying GD or any GD variant (mini-batch GD).

Bad feature skew

In the following two-feature example you can see a *deformed* loss curve. The gradient is more pronounced in one direction, so we require more iterations to reach the minimum. This is because one feature (feat1) has a larger range and spread than the other (feat2). Notice that the loss curve is *squeezed* along the w_1 axis, and *stretched* along the w_2 axis.

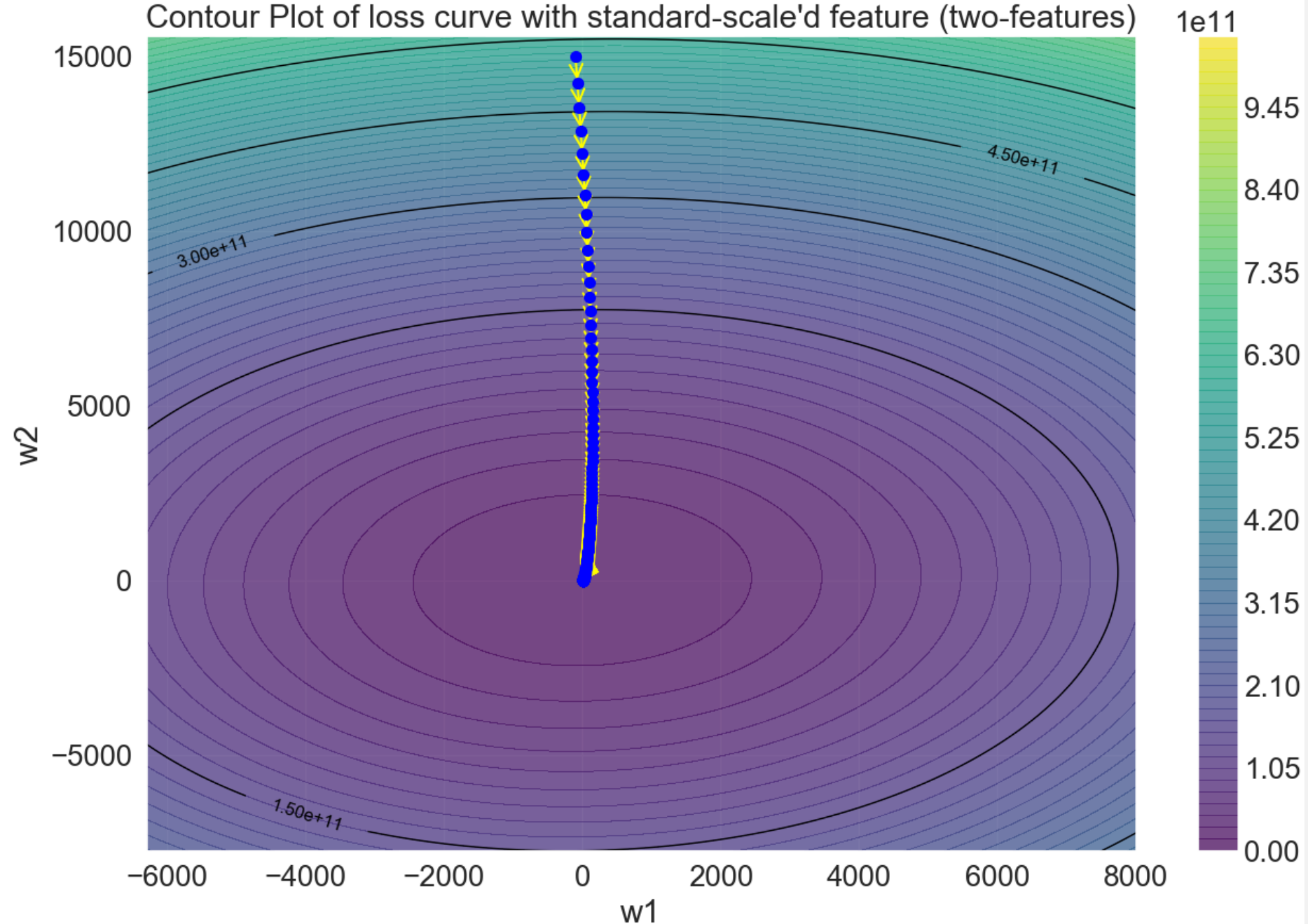
This is GD after 500 iterations with learning rate of 0.001.



Fixing feature skew

If we applied `StandardNormalScaler` to the features, we get this much *better* loss curve. Notice it is not skewed in one weight axis over the other.

GD took 276 iterations to reach minimum with same learning rate of 0.001.



MinMaxScaler or StandardNormalScaler?

Although normalization via min-max scaling is a commonly used technique that is useful when we need values in a bounded interval, standardization can be more practical for many machine learning algorithms, especially for optimization algorithms such as gradient descent. The reason is that many linear models, such as the logistic regression and SVM that we remember from *Chapter 3, A Tour of Machine Learning Classifiers Using scikit-learn*, initialize the weights to 0 or small random values close to 0. Using standardization, we center the feature columns at mean 0 with standard deviation 1 so that the feature columns takes the form of a normal distribution, which makes it easier to learn the weights. Furthermore, standardization maintains useful information about outliers and makes the algorithm less sensitive to them in contrast to min-max scaling, which scales the data to a limited range of values.

**From "Python Machine Learning"
by Sebastian Raschka pg122**

StandardScaler and SGDRegressor in scikit-learn

```
from sklearn.linear_model import SGDRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# we'll use scikit-learn pipeline for ease-of-use
pipeline = make_pipeline(StandardScaler(),
                        SGDRegressor(max_iter=1000))
pipeline.fit(X_train, y_train)

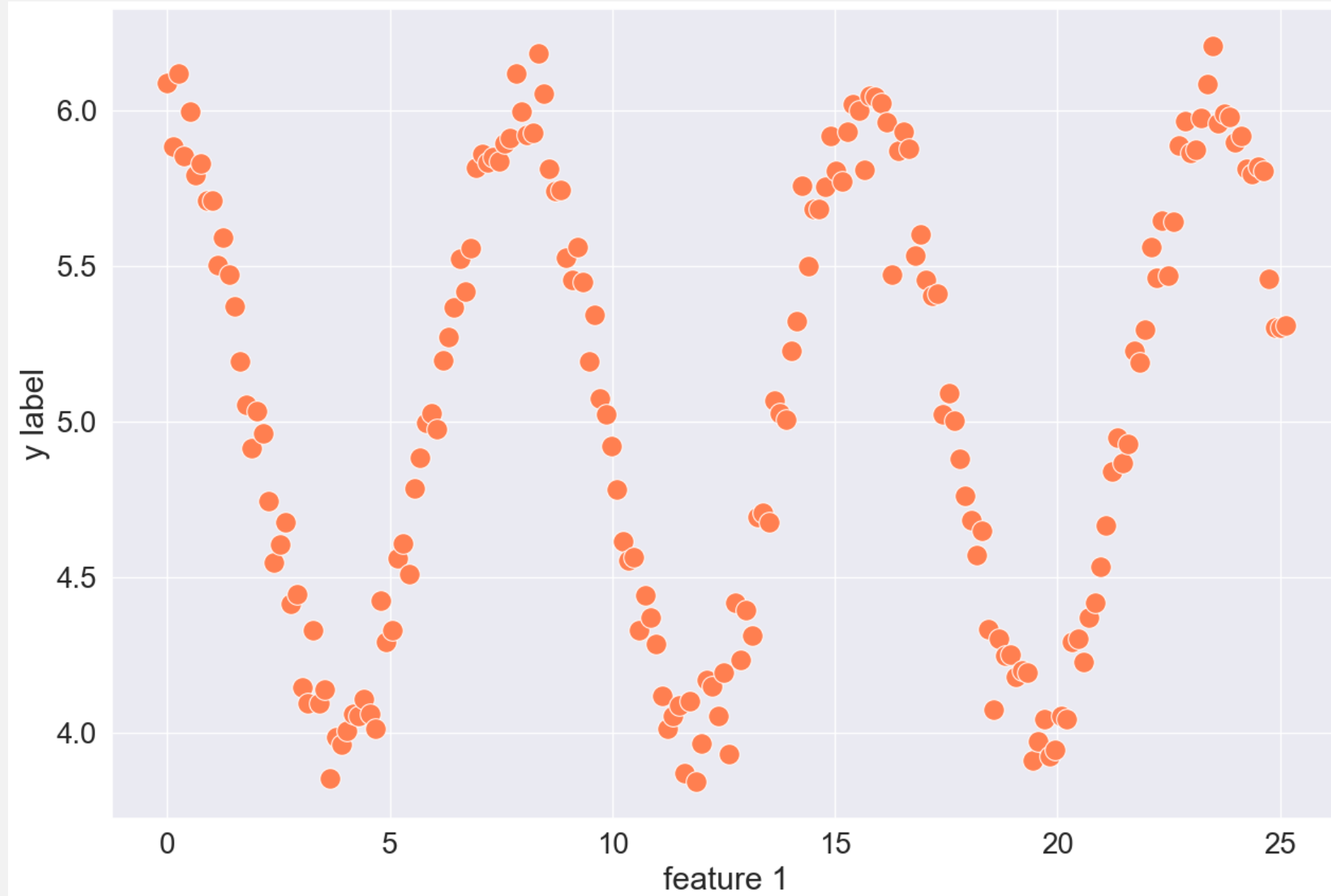
# predict will scale, then predict using model
pipeline.predict(X_test)
```

- Recall that scikit-learn's [LinearRegression](#) computes the closed-form solution (does not use gradient descent).
- [SGDRegressor](#) does SGD (batch size of 1). Also, has [partial_fit](#)(X_batch, y_batch) for you to use your own batch updates.
- Note above we use scikit-learn [pipeline](#) for ease-of-use.

Polynomial (Linear) Regression

Example of non-linear relationship between input and output [1]

- Consider the following one-feature training data. The relationship between the feature and output is clearly not linear.

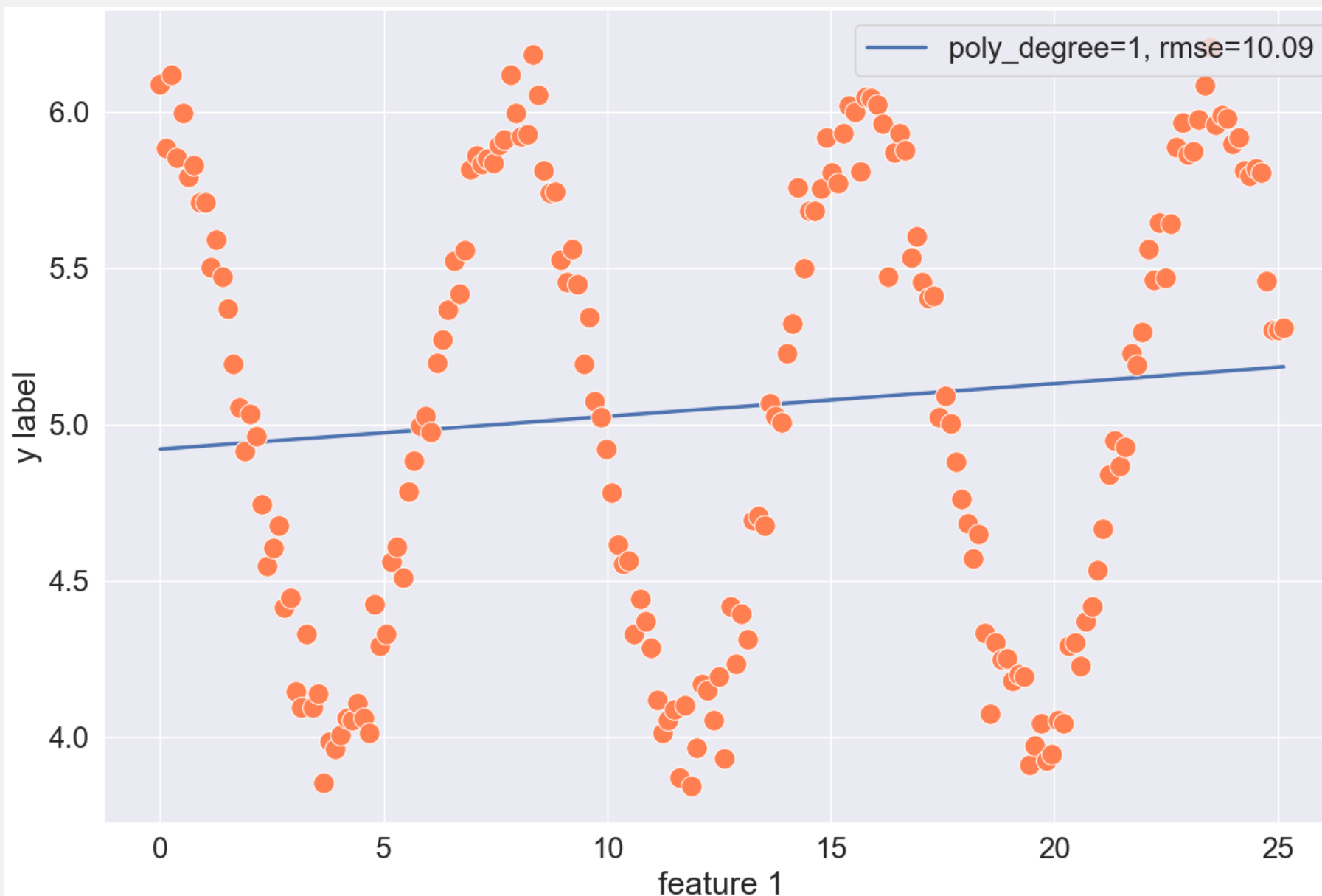


Example of non-linear relationship between input and output [2]

- We can fit a line to this data, but the line fit is not that good.
- Root-mean-square-error (RMSE) is just the square root of the average squared loss.

$$RMSE = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2}$$

- Can we use a polynomial hypothesis to get a better fit? How do we do this?



Example of non-linear relationship between input and output [3]

- Can we use a polynomial hypothesis to get a better fit? How do we do this?
- Denote the single feature as *feat*

- Degree 2 polynomial

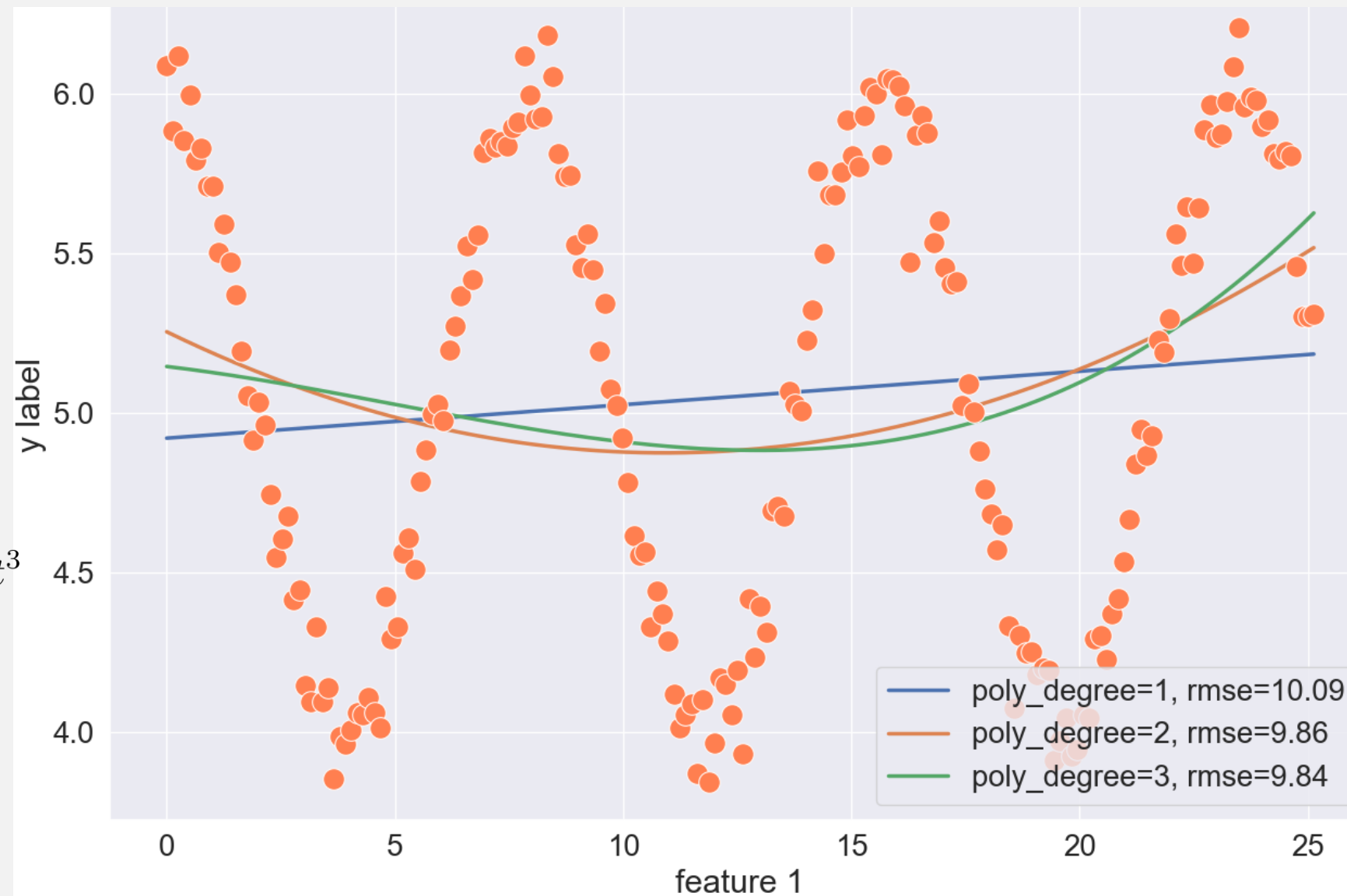
$$\hat{y} = w_0 + w_1 feat + w_2 feat^2$$

- Degree 3 polynomial

$$\hat{y} = w_0 + w_1 feat + w_2 feat^2 + w_3 feat^3$$

- Degree p polynomial

$$\hat{y} = w_0 + \sum_{d=1}^p w_d feat^d$$



One-Feature Polynomial Regression

- For a one-feature problem and polynomial regression of degree p , the training set data matrix X looks as follows:

$$X = \begin{bmatrix} 1 & x_{11} & x_{11}^2 & \dots & x_{11}^p \\ 1 & x_{21} & x_{21}^2 & \dots & x_{21}^p \\ & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m1}^2 & \dots & x_{m1}^p \end{bmatrix}$$

- Note that the single feature column x_{i1} is transformed into multiple columns; where each column is raised to a degree. Each column is treated as an independent feature in linear regression.

2-feature Polynomial Regression

- For an 2-feature problem and polynomial regression of degree 3.

$$h(x_i) = w_0 + \underbrace{w_1x_{i1} + w_2x_{i2}}_{\text{degree 1}} + \underbrace{w_3x_{i1}x_{i2} + w_4x_{i1}^2 + w_5x_{i2}^2}_{\text{degree 2}} + \underbrace{w_6x_{i1}^2x_{i2} + w_7x_{i1}x_{i2}^2 + w_8x_{i1}^3 + w_9x_{i2}^3}_{\text{degree 3}}$$

Terms with more than one feature
interacting are called interactions

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & x_{11}x_{12} & x_{11}^2 & x_{12}^2 & x_{11}^2x_{12} & x_{11}x_{12}^2 & x_{11}^3 & x_{12}^3 \\ 1 & x_{21} & x_{22} & x_{21}x_{22} & x_{21}^2 & x_{22}^2 & x_{21}^2x_{22} & x_{21}x_{22}^2 & x_{21}^3 & x_{22}^3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_{m1} & x_{m2} & x_{m1}x_{m2} & x_{m1}^2 & x_{m2}^2 & x_{m1}^2x_{m2} & x_{m1}x_{m2}^2 & x_{m1}^3 & x_{m2}^3 \end{bmatrix}$$

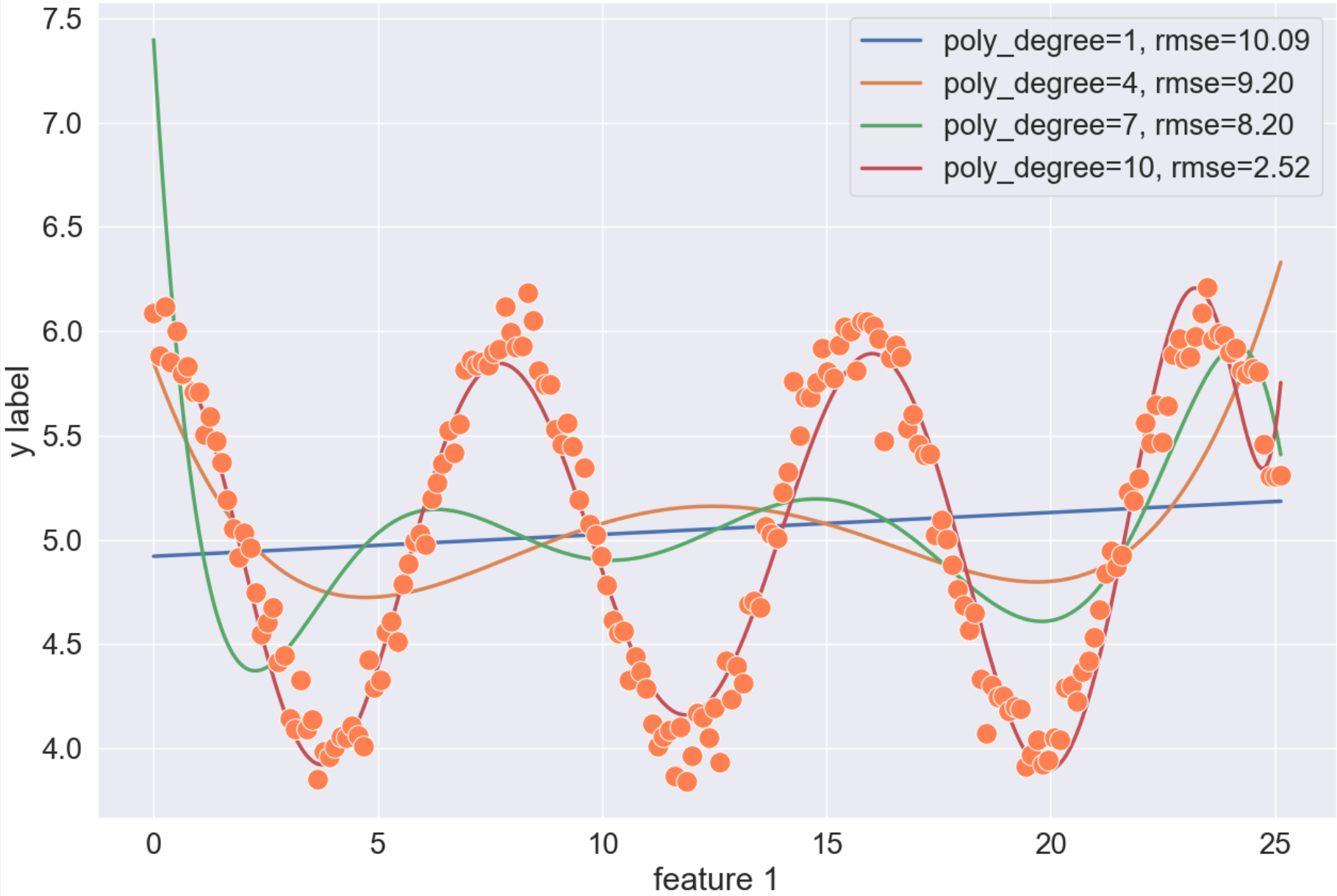
Scikit-learn Polynomial Feature generator

```
class sklearn.preprocessing.PolynomialFeatures(degree=2, *,  
interaction_only=False, include_bias=True, order='C') \[source\]
```

```
from sklearn.preprocessing import PolynomialFeatures  
from sklearn.linear_model import LinearRegression  
  
pf_generator = PolynomialFeatures(degree=4)  
  
X_train_poly = pf_generator.fit_transform(X_train)  
  
pr_model = LinearRegression(fit_intercept=False)  
pr_model.fit(X_train_poly, y_train)  
  
X_test_poly = pf_generator.fit_transform(X_test)  
pr_model.predict(X_test_poly)
```

- [PolynomialFeatures](#) takes in the original feature matrix (without the 1 column) and generates polynomial feature matrix up to the specified degree. (see short guide [here](#))
- Can also use it as part of a pipeline.
- Note: we set `fit_intercept=False` since `PolynomialFeatures` will add 1-column by default (`include_bias=True`).

Example of non-linear relationship between input and output [4]



Remarks on Polynomial Regression

- Polynomial regression is still considered linear regression. This is because the hypothesis is linear in the weights; although non-linear in the features.

$$h(x_i) = w_0 + w_1x_{i1} + w_2x_{i2} + w_3x_{i1}x_{i2} + w_4x_{i1}^2 + w_5x_{i2}^2 + w_6x_{i1}^2x_{i2} + w_7x_{i1}x_{i2}^2 + w_8x_{i1}^3 + w_9x_{i2}^3$$

- An example of non-linearity in the weights would be something like this:

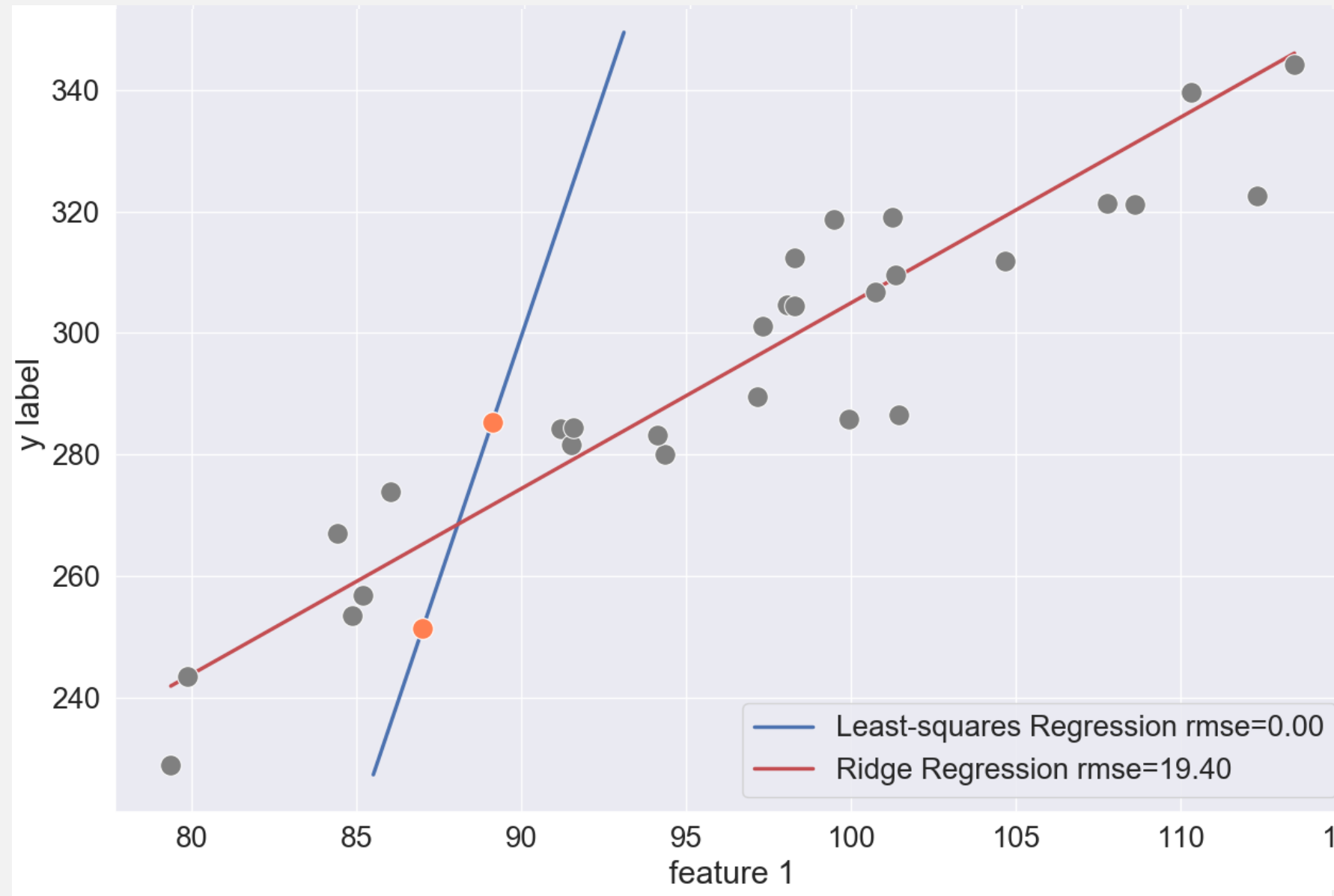
$$\hat{y} = w_0w_1w_2 + \frac{w_1}{w_2w_3}feat1 + w_4e^{w_3^2}feat2$$

- You can generalize this idea to not just polynomial transformations of the features, but any non-linear transformation of the features like $\sin(feats)$, $\cos(feats)$, $\tan(feats)$, etc.

Regularization – Ridge, Lasso, and Elastic-net Regression

Use of Ridge Regression to combat overfitting

- Consider the following one-feature data.
- The orange dots represent the training data (small dataset).
- The grey dots represents **unseen** data distribution.
- Least-squares regression gets a perfect fit on the training data. But bad fit on the underlying data. It is overfitting.
- To combat overfitting, we can use Ridge Regression which is represented by the red-line. This line gets a slightly worse fit on the training data, but a better fit on the unseen data.



Regularization [1]

- In general, *complex* hypothesis spaces are prone to overfitting.
- We can apply restrictions on our hypothesis space to make it *simpler*.
- **Regularization** refers to techniques that apply restrictions on ML algorithms to combat overfitting (make their hypothesis space *simpler*).
- One common regularization technique is to apply restrictions on the weight parameters.
- The general formula for weight regularization is to apply a cost term to the weights $R(w)$. This cost term is then added to our loss function with a scaling factor $\lambda \in [0, \infty)$:

$$\ell(w) = \sum_{i=1}^m (y_i - x_i^T w)^2 + \lambda R(w)$$

$$w^* = \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - x_i^T w)^2 + \lambda R(w)$$

$$\ell(w) = \sum_{i=1}^m (y_i - x_i^T w)^2 + \lambda R(w)$$

- λ is used to tradeoff how much we want to restrict our weights to combat overfitting versus getting a better fit on the training data. When $\lambda = 0$ there is no regularization which is vanilla least-squares regression. When $\lambda \rightarrow \infty$ then the weights are forced to 0.
- How to find the right value for λ ? Use a separate validation set or k -fold cross-validation; a topic we'll discuss later in the course.
- Popular cost terms for $R(w)$: Ridge regression, LASSO regression, Elastic-net regression.

- In Ridge regression, the cost term called L2-norm $R(w) = ||w||_2^2 = \sum_j w_j^2$.
- Thus, the loss function becomes:

$$\ell(w) = \sum_{i=1}^m (y_i - x_i^T w)^2 + \lambda \sum_j w_j^2$$

$$w^* = \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - x_i^T w)^2 + \lambda \sum_j w_j^2$$

- Ridge regression promotes weight vectors with small L2-norm. Helps to combat overfitting by reducing the effect of outliers or when we have little data.

$$w^* = \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - x_i^T w)^2 + \lambda \sum_j w_j^2$$

- This can be solved by taking the gradient and setting it to 0. Then solving for w .

$$\begin{aligned} \frac{\partial \ell(w)}{\partial w} &= -2X^T Y + 2X^T X w + 2\lambda w \\ &= -2X^T Y + 2(X^T X + \lambda I)w = \mathbf{0} \end{aligned} \quad \text{Note that } \lambda w = (\lambda I)w$$

$$\implies (X^T X + \lambda I)w = X^T Y$$

$$\implies w^* = (X^T X + \lambda I)^{-1} X^T Y$$

- This is the closed form solution for Ridge Regression.

$$\begin{aligned}\frac{\partial \ell(w)}{\partial w} &= -2X^T Y + 2X^T X w + 2\lambda w \\ &= -2X^T Y + 2(X^T X + \lambda I)w\end{aligned}$$

- Alternatively, we can perform gradient descent to find w^* using the update rule:

$$\begin{aligned}\hat{w} &:= \hat{w} - \alpha \nabla_w \ell(\hat{w}) \\ &:= \hat{w} - \alpha [X^T (X\hat{w} - Y) + \lambda \hat{w}]\end{aligned}$$

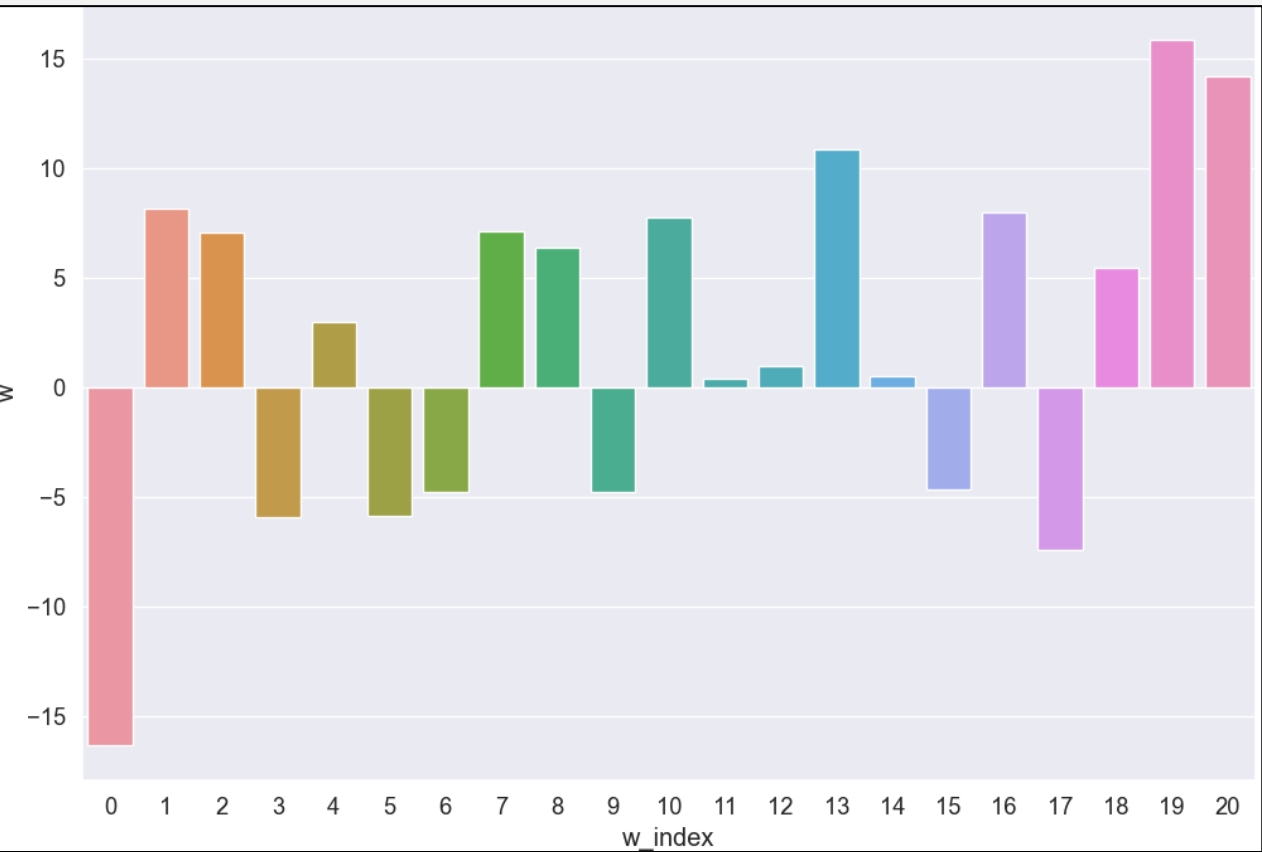
- In LASSO regression, the cost term called L1-norm $R(w) = ||w||_1 = \sum_j |w_j|$.
- Thus, the loss function becomes:

$$\ell(w) = \sum_{i=1}^m (y_i - x_i^T w)^2 + \lambda \sum_j |w_j|$$

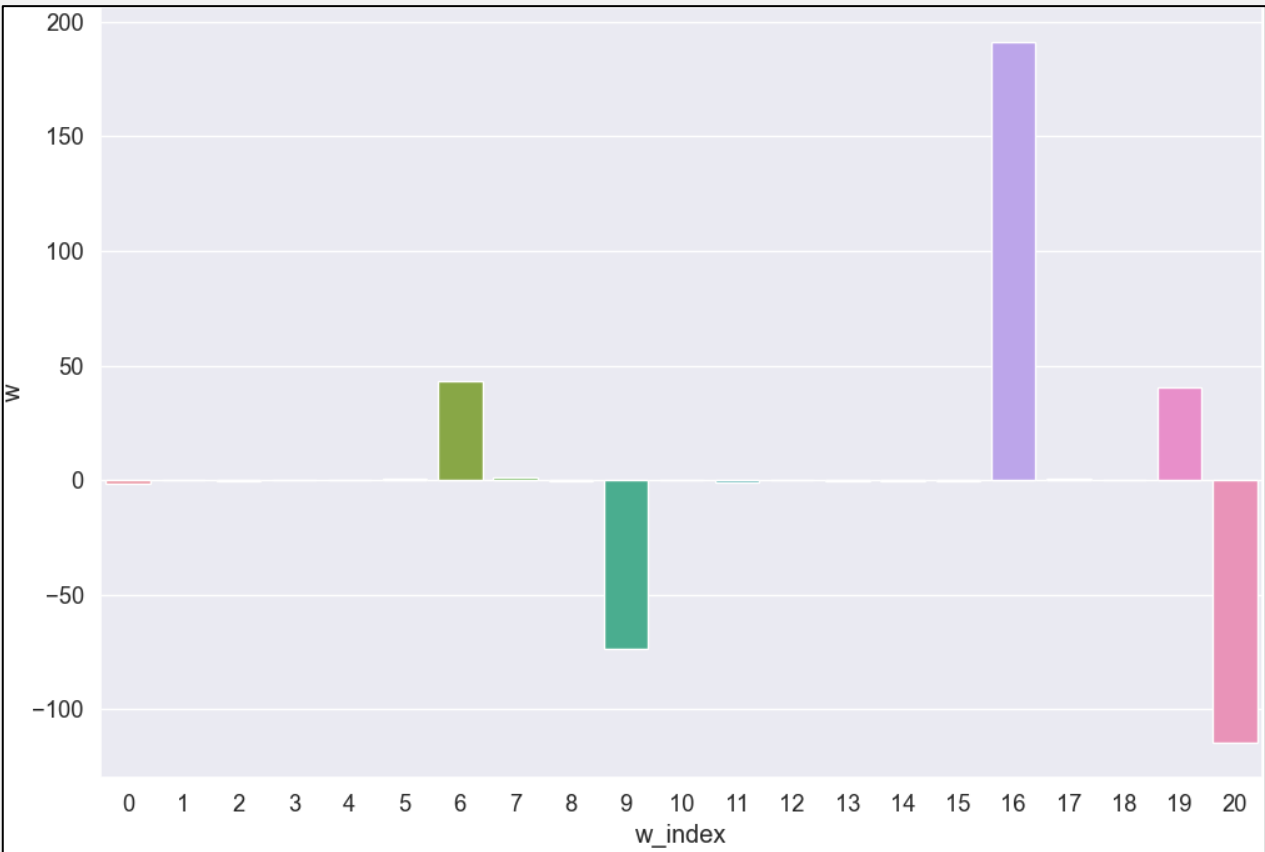
$$w^* = \arg \min_{w \in \mathbb{R}^{n+1}} \sum_{i=1}^m (y_i - x_i^T w)^2 + \lambda \sum_j |w_j|$$

- LASSO regression promotes sparse weight vectors; that is the weight vector has only a few non-zero elements. Can be helpful if you know from prior information that only a few features are important.

LASSO Regression Weight Visualizations



Randomly initialized weights before training



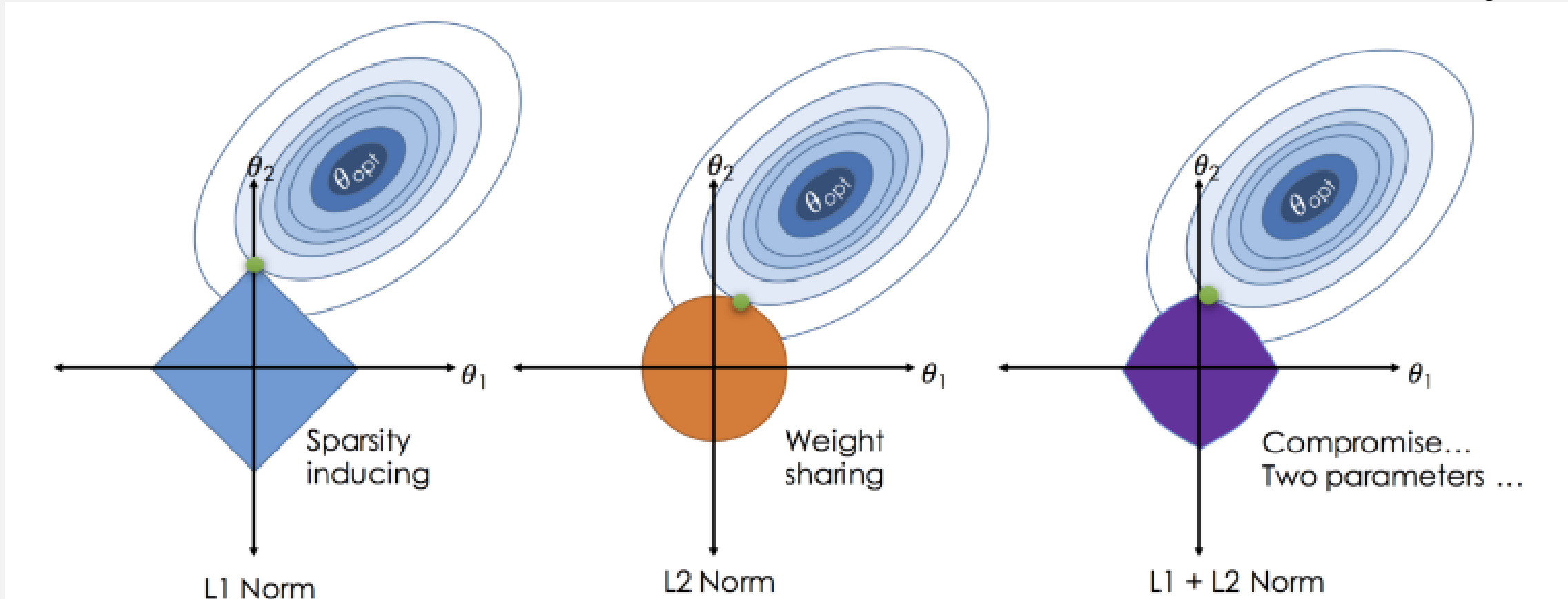
Weights after training

- No closed form solution due the problematic derivative of the absolute value term.
- Cannot directly use gradient-descent due the problematic absolute value term.
- However, as mentioned there are other techniques to solve optimization problems. For Lasso, can be solved using subgradient methods and proximal gradient methods. We will not cover them.
- For this course, just need to know that it promotes another form of regularization; namely promoting sparse weight vectors. This is just another way of combatting overfitting.
- Setting λ can also be done by using a validation set or k -fold cross-validation; a topic we'll discuss later in the course.

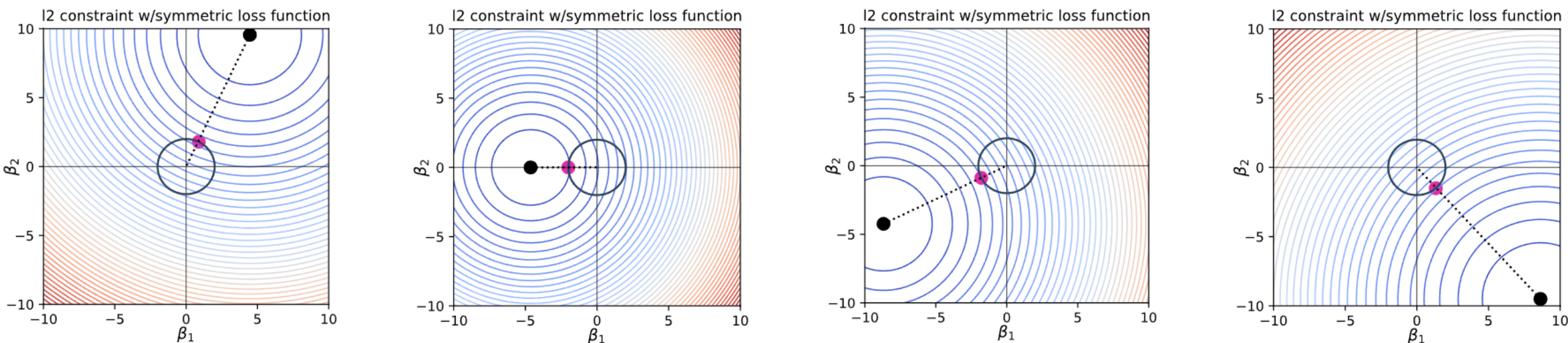
- In Elastic Net regression, the cost term combines ridge and LASSO regression. Thus, the loss function becomes:

$$\ell(w) = \sum_{i=1}^m (y_i - x_i^T w)^2 + \lambda_1 \sum_j |w_j| + \lambda_2 \sum_j w_j^2$$

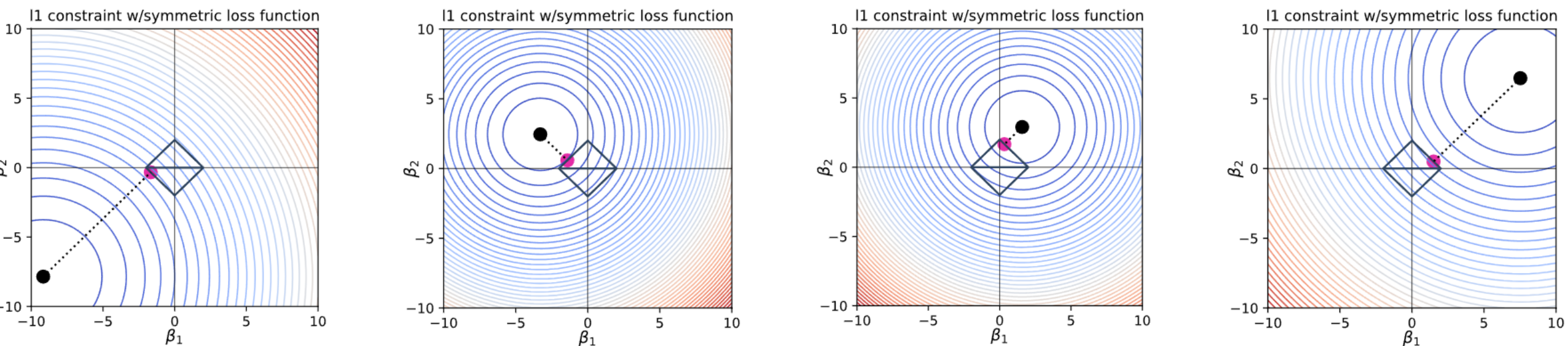
- By playing around with λ_1 and λ_2 we are promoting sparsity (LASSO) and small norm solutions (Ridge).



- We have a two-feature problem with weights θ_1 and θ_2 . The blue contours is the squared-loss on the training set. It is the same in all three plots.
- These three plots also show the **cost contours** for L1 norm (LASSO), L2 norm (Ridge), and L1+L2 (Elastic net) for the weights θ_1 and θ_2 .
- As can be seen, the L1 norm promotes sparse solution via intersection of squared loss with tip of the L1 diamond cost.
- The L2 norm promotes small norms by intersection of the squared loss with the L2 ball cost.
- Finally, L1+L2 norm compromises between the two norm costs.
- Higher values for λ promotes focusing on reducing the weight-cost over the squared-loss. With $\lambda \rightarrow \infty$ promoting setting the weights to 0.



- Illustration of various Ridge loss scenarios for two weight parameters.
- L2 norm promotes small norms by intersection of the squared loss with the L2 ball cost.
- Higher values for λ promotes focusing on reducing the weight-cost over the squared-loss. With $\lambda \rightarrow \infty$ promoting setting the weights to 0.



- Illustration of various LASSO loss scenarios for two weight parameters.
- L1 norm promotes sparse solution via intersection of squared loss with tip of the L1 diamond cost.
- Higher values for λ promotes focusing on reducing the weight-cost over the squared-loss. With $\lambda \rightarrow \infty$ promoting setting the weights to 0.

- These methods are implemented in scikit-learn.
- [sklearn.linear_model.Ridge](#)
- [sklearn.linear_model.Lasso](#)
- [sklearn.linear_model.ElasticNet](#)
- Note: scikit-learn by default assume you didn't add 1 to your data. See the documentation of *fit_intercept* argument.

```
1 from sklearn.linear_model import Ridge
2
3 clf = Ridge(alpha=1.0) # alpha here is the lambda | in the slides
4 clf.fit(X, y)
```

Remarks on Ordinary LR vs Regularization

- Ordinary LR is ideal when the relationship between features and output is linear, and the number of instances is much larger than number of features. (tall data matrix)
- If we have more features than instances (wide data matrix), this tends to overfit the training data. So, in this case, we can apply regularization to combat overfitting.
- For example, if we had $m = 18$ training examples and $n = 20$ features, regularization can help get a linear fit that doesn't overfit the training data.
- You may also use LASSO if you have a lot of features but suspect that only few features are useful for the output.

LASSO Regression with Polynomials

- Suppose we want to apply polynomial regression to a learning task. We specify the degree p .
- We can combat overfitting by applying LASSO regularization to promote sparsity.
- Recall that this will push some of the feature weights to zero. In effect, some of the polynomial features will be *zero'd* out.

$$\begin{aligned}\hat{y}_i = & w_0 + w_1x_{i1} + w_2x_{i1}^2 + \cdots + w_px_{i1}^p + \cdots \\ & + w_{p+1}x_{12} + w_{p+2}x_{12}^2 + \cdots + w_{2p}x_{12}^p + \cdots + w_{np}x_{1n}^p\end{aligned}$$

- Thus, regularization can be used to restrict a complex polynomial model of degree p . This in turn makes the hypothesis space simpler which helps combat overfitting.